

Redes de Comunicação

2026/27

Camada de Aplicação

Capítulo 2: Camada de aplicação

- ❑ 2.1 Princípios das aplicações em rede
- ❑ 2.2 Web e HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Correio electrónico
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS
- ❑ 2.6 Aplicações P2P
- ❑ 2.7 Programação com *sockets* — TCP
- ❑ 2.8 Programação com *sockets* — UDP

Capítulo 2: Camada de aplicação

Objectivos:

- ❑ aspectos conceptuais e de implementação de protocolos de aplicação em rede
 - ❖ modelos de serviço da camada de transporte
 - ❖ paradigma cliente-servidor
 - ❖ paradigma *peer-to-peer*
- ❑ aprender os conceitos dos protocolos de aplicação examinando alguns mais comuns
 - ❖ HTTP
 - ❖ FTP
 - ❖ SMTP / POP3 / IMAP
 - ❖ DNS
- ❑ programar aplicações em rede
 - ❖ API de *sockets*

Algumas aplicações em rede

- ❑ *e-mail*
- ❑ *web*
- ❑ *instant messaging*
- ❑ *terminal remoto*
- ❑ *partilha de ficheiros P2P*
- ❑ *jogos multi-utilizador em rede*
- ❑ *streaming de vídeo*
- ❑ *voz sobre IP*
- ❑ *videoconferência em tempo real*
- ❑ *computação grid*
- ❑ *redes sociais*
- ❑ *...*

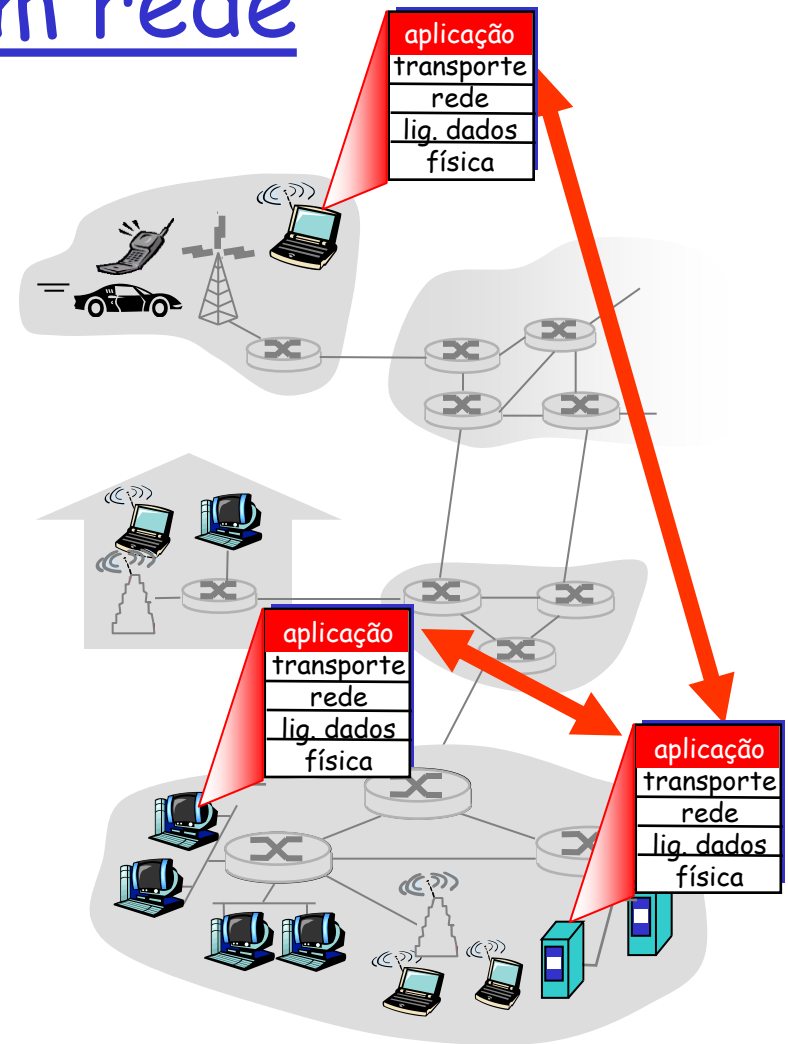
Criar uma aplicação em rede

escrever programas que

- ❖ correm nos terminais
- ❖ comunicam através da rede
- ❖ e.g., o *browser* comunica com o servidor *web*

pouco software é escrito para os dispositivos no núcleo da rede

- ❖ estes dispositivos não correm aplicações de utilizador
- ❖ manter as aplicações nos terminais permite agilizar o seu desenvolvimento e divulgação



Capítulo 2: Camada de aplicação

- ❑ 2.1 Princípios das aplicações em rede
- ❑ 2.2 Web e HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Correio electrónico
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS
- ❑ 2.6 Aplicações P2P
- ❑ 2.7 Programação com sockets — TCP
- ❑ 2.8 Programação com sockets — UDP

Arquitecturas aplicacionais

- ❑ Cliente-servidor
- ❑ *Peer-to-peer* (P2P)
- ❑ Híbridas de cliente-servidor e P2P

Arquitetura cliente-servidor



servidor:

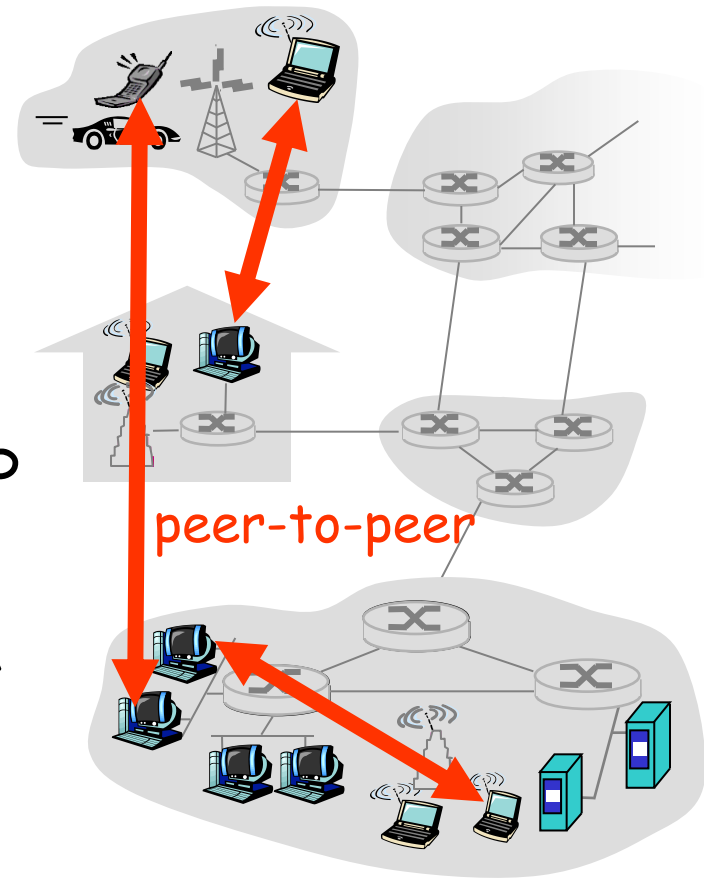
- ❖ sempre ligado
- ❖ endereço IP fixo
- ❖ replicação de servidores para escalabilidade (custo)

cliente:

- ❖ comunica com o servidor
- ❖ não precisa de estar sempre ligado
- ❖ pode ter endereço IP dinâmico
- ❖ não comunica directamente com outros clientes

Arquitectura P2P pura

- ❑ sem servidores dedicados
- ❑ terminais comunicam directamente entre si
- ❑ podem não estar sempre ligados
- ❑ podem mudar de endereço IP
- ❑ **muito escalável**
 - ❖ aumento simultâneo na procura e na oferta
- ❑ **difícil de gerir**
 - ❖ nós ligados intermitentemente
 - ❖ endereços dos nós mudam



Arquitectura híbrida

BitTorrent

- ❖ aplicação P2P para partilha de ficheiros
- ❖ servidor central (*tracker*) permite descobrir endereços IP de *peers* a participar na partilha
- ❖ troca directa de pedaços do ficheiro *peers* (sem intervenção do *tracker*)

Instant messaging

- ❖ mensagens de texto enviadas directamente (P2P)
- ❖ serviço central de detecção de presença e localização
 - o utilizador regista o seu endereço IP no servidor central quando se liga
 - usa o servidor central para descobrir estado e endereço IP dos seus contactos

Comunicação entre processos

Processo: instância dum programa a correr numa máquina

- dentro da mesma máquina, dois processos comunicam usando os mecanismos de **IPC** do sistema operativo
- processos em máquinas diferentes comunicam através da troca de **mensagens**

Cliente: processo que inicia activamente a comunicação

Servidor: processo que espera passivamente ser contactado

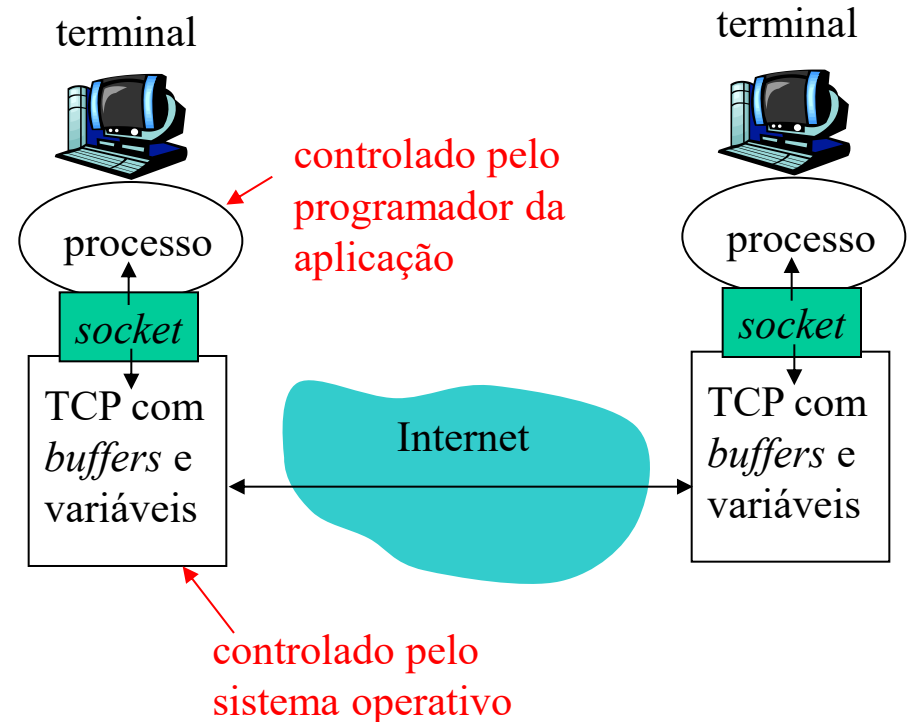
- Nota: as aplicações P2P funcionam simultaneamente como cliente e servidor

Endereçamento de processos

- ❑ para poder receber mensagens, um processo deve ter um **identificador** (endereço)
- ❑ cada terminal tem um endereço IP de 32 bits
- ❑ **Questão:** será o endereço IP do terminal suficiente para identificar de forma unívoca o processo?
Resposta: não, porque há múltiplos processos a correr no mesmo terminal
- ❑ o identificador inclui tanto o **endereço IP** como o **número da porta** associada ao processo no terminal
- ❑ Exemplos de portas:
 - ❖ Servidor HTTP: 80
 - ❖ Servidor de email: 25
- ❑ para enviar um pedido HTTP ao servidor web `www.dcc.fc.up.pt`:
 - ❖ **Endereço IP:** 193.136.39.12
 - ❖ **Porta:** 80

Sockets

- ❑ um processo envia e recebe mensagens através duma **socket**
- ❑ escrita numa **socket**
 - ❖ indica ao S.O. que deve enviar a mensagem
 - ❖ assume a existência de uma infraestrutura de transporte que leva a mensagem até ao processo receptor
- ❑ API permite (1) escolher o protocolo de transporte e (2) alterar alguns parâmetros



Protocolo de aplicação define

- ❑ **Tipos** de mensagens trocadas

- ❖ e.g., pedido, resposta

- ❑ **Sintaxe** das mensagens

- ❖ que campos contêm, como se separam esses campos

- ❑ **Semântica** das mensagens

- ❖ significado da informação contida nos diferentes campos

- ❑ **Regras** de como e quando os processos devem reagir às mensagens

Protocolos de domínio público:

- ❑ definidos em RFCs

- ❑ possibilitam interoperação entre sistemas

- ❑ e.g., HTTP, SMTP

Protocolos proprietários:

- ❑ só tem acesso a eles quem o detentor dos direitos decidir

- ❑ e.g., Zoom

De que serviço de transporte precisa uma aplicação?

Perdas

- ❑ algumas aplicações (e.g., áudio) toleram algumas perdas
- ❑ outras (e.g., transferência de ficheiros) exigem uma transferência 100% fiável

Latência

- ❑ algumas aplicações (e.g., voz sobre IP, jogos em rede) exigem atrasos baixos para funcionar aceitavelmente

Débito

- ❑ algumas aplicações (e.g., multimédia) precisam de um débito mínimo
- ❑ outras (designadas aplicações elásticas) usam aquilo que tiverem disponível

Segurança

- ❑ integridade dos dados, privacidade (cifragem), autenticação das partes

Requisitos de transporte de algumas aplicações

Aplicação	Perdas	Débito	Sensível aos atrasos
transf. de ficheiros	não tolera	elástica	não
e-mail	não tolera	elástica	não
web	não tolera	elástica	não
áudio/vídeo em tempo-real	tolera	áudio: 5kbps-1Mbps vídeo: 10kbps-5Mbps	sim, 100s ms
áudio/vídeo armazen.	tolera	idem	sim, alguns seg.
jogos interactivos	tolera	desde alguns kbps	sim, 100s ms
<i>instant messaging</i>	não tolera	elástica	parcialmente

Serviço dos protocolos de transporte na Internet

Serviço TCP:

- ❑ *orientado a conexões*: exige estabelecimento prévio de conexão entre o emissor e o receptor
- ❑ *transporte fiável* entre os processos emissor e receptor
- ❑ *controlo de fluxo*: o emissor não envia mais rápido do que o receptor consegue processar
- ❑ *controlo de congestionamento*: redução de débito quando a rede está sobrecarregada
- ❑ *não tem* delineação de mensagens nem garantias de atraso máximo ou débito mínimo

Serviço UDP:

- ❑ *transporte não-fiável* entre os processos emissor e receptor
- ❑ *delineação de mensagens*: um datagrama UDP corresponde a uma mensagem
- ❑ *não tem* estabelecimento prévio de conexões, fiabilidade, controlo de fluxo, controlo de congestionamento, garantias de atraso máximo ou de débito mínimo
 - ❖ Nem todas as aplicações necessitam de tudo isto...

Protocolos de aplicação e de transporte na Internet

Aplicação	Protocolo da camada de aplicação	Protocolo de transporte usado
<i>e-mail</i>	SMTP [RFC 2821]	TCP
terminal remoto	Telnet [RFC 854]	TCP
<i>web</i>	HTTP [RFC 2616]	TCP
transf. de ficheiros	FTP [RFC 959]	TCP
<i>streaming</i> multimédia	HTTP (e.g., YouTube), RTP [RFC 1889]	TCP ou UDP
voz sobre IP	SIP, RTP, proprietários (e.g., Zoom)	tipicamente UDP

Segurança

TCP e UDP

- ❑ sem cifragem
- ❑ *passwords* em *cleartext* visíveis na Internet

SSL/TLS

- ❑ cifragem/privacidade
- ❑ integridade dos dados
- ❑ autenticação das extremidades

SSL/TLS funciona na camada de aplicação

- ❑ Biblioteca SSL "fala" com o TCP

API de sockets SSL

- ❑ *passwords* em *cleartext* circulam na Internet cifradas
- ❑ certificados garantem identidade do servidor

O SSL é obsoleto e foi substituído pelo TLS

Capítulo 2: Camada de aplicação

- ❑ 2.1 Princípios das aplicações em rede
- ❑ 2.2 Web e HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Correio electrónico
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS
- ❑ 2.6 Aplicações P2P
- ❑ 2.7 Programação com sockets — TCP
- ❑ 2.8 Programação com sockets — UDP

Web e HTTP

Alguna terminologia

- ❑ Uma **página web** consiste num **ficheiro-base HTML** que inclui uma série de **objectos** referenciados
- ❑ Os objectos podem ser ficheiros HTML, imagens JPEG, ficheiros de som, vídeos, ...
- ❑ Cada objecto é endereçável por um **URL**
- ❑ Exemplo de URL:

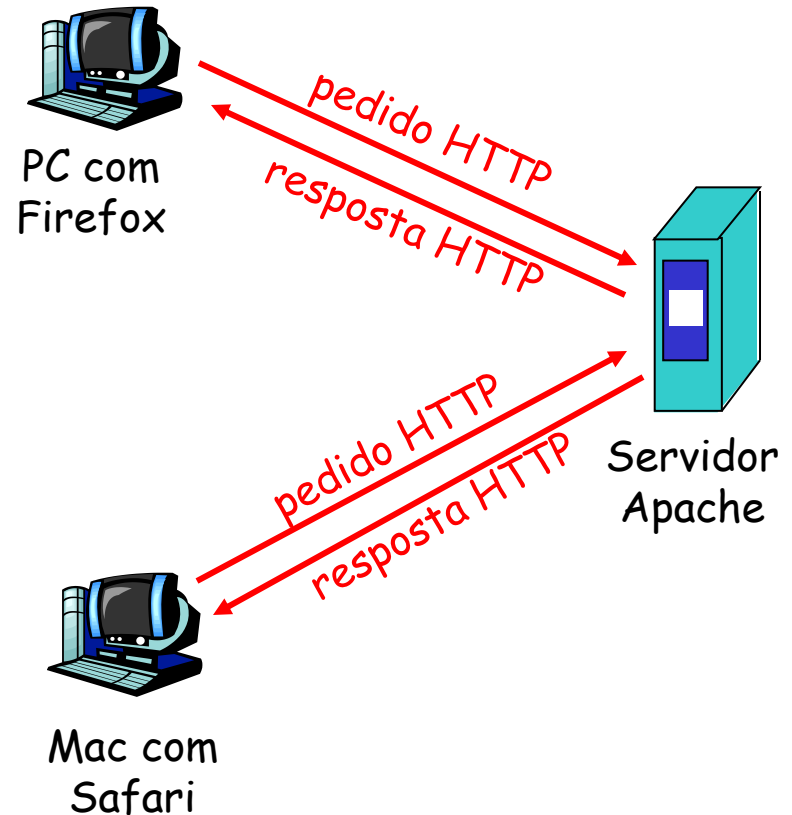
`http://www.dcc.fc.up.pt/~rprior/photo.jpg`

protocolo servidor caminho

HTTP — Panorama geral

HTTP: hypertext transfer protocol

- ❑ Protocolo de camada de aplicação na web
- ❑ modelo cliente/servidor
 - ❖ *cliente*: browser que pede, recebe e mostra os objectos ao utilizador
 - ❖ *servidor*: envia os objectos em resposta aos pedidos do browser
- ❑ HTTP 1.0: RFC 1945
- ❑ HTTP 1.1: RFC 2068
- ❑ HTTP 2: RFC 9113
- ❑ HTTP 3: RFC 9114



HTTP — Panorama geral (cont.)

Usa o TCP:

- ❑ o cliente inicia a conexão TCP para a porta 80 do servidor
- ❑ o servidor aceita a conexão TCP do cliente
- ❑ são trocadas mensagens HTTP (camada de aplicação) entre o *browser* (cliente HTTP) e o servidor *web* (servidor HTTP)
- ❑ a conexão TCP é fechada

HTTP é "stateless"

- ❑ o servidor não mantém qualquer informação (estado) sobre os pedidos anteriores do cliente

nota

Protocolos com manutenção de estado são complexos!

- ❑ memória para estado
- ❑ se o servidor ou o cliente crasham, as suas visões do estado podem ficar inconsistentes e ter de ser ressincronizadas

Persistência das conexões HTTP

HTTP não-persistente

- ❑ Numa conexão TCP só pode ser transmitido um único objecto
- ❑ Usado no HTTP/1.0

HTTP persistente

- ❑ Podem enviar-se múltiplos objectos numa única conexão TCP
- ❑ Necessidade de definir onde termina cada mensagem
- ❑ HTTP/1.1 normalmente usa conexões persistentes

HTTP não-persistente

O utilizador introduz o URL

`www.dcc.fc.up.pt/~rprior/homepage/index.html` (contém texto e referências para 10 imagens jpeg)

1a. O cliente HTTP inicia uma conexão TCP para o (processo) servidor na porta 80 do terminal `www.dcc.fc.up.pt`

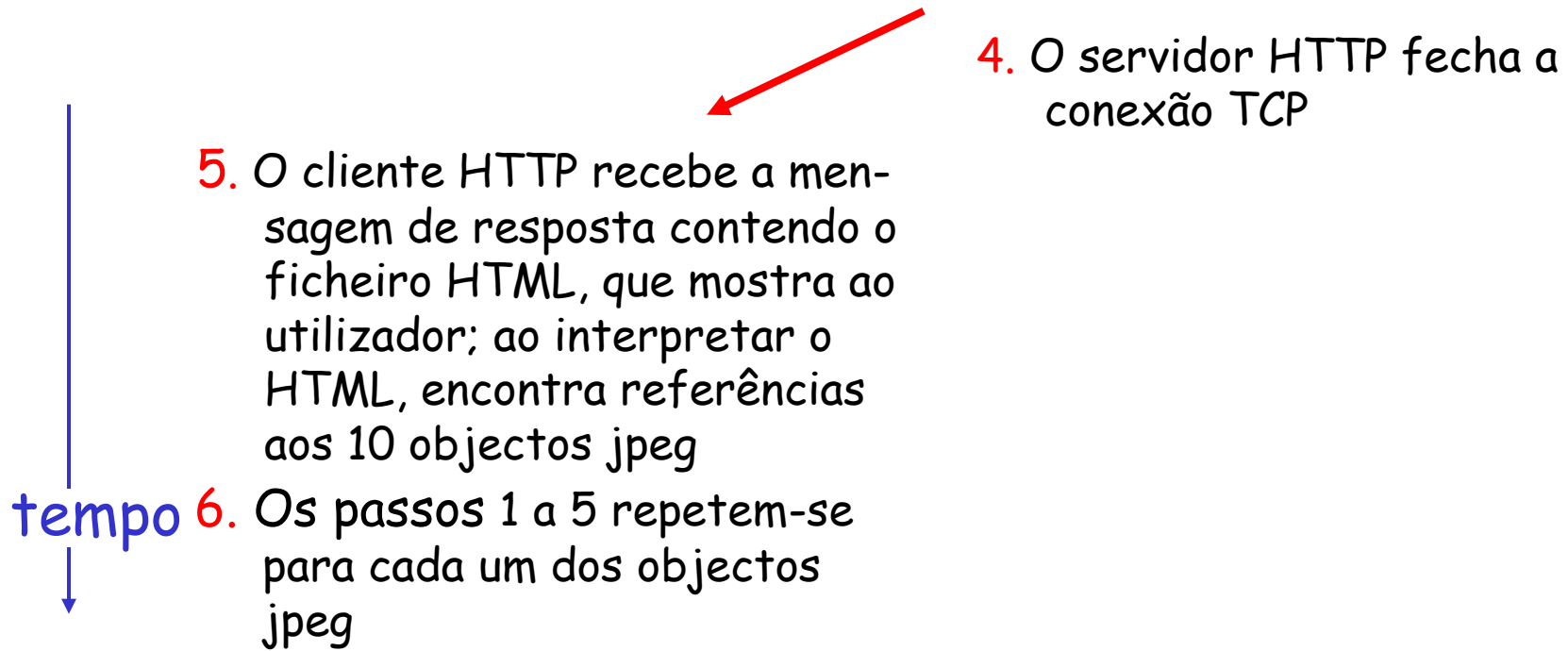
1b. O servidor HTTP no terminal `www.dcc.fc.up.pt` estava à espera de conexões TCP na porta 80; aceita a conexão notificando o cliente

2. O cliente HTTP envia uma *mensagem de pedido* na socket com um URL que indica que o cliente pretende o objecto `/~rprior/hompaga/index.html`

3. O servidor HTTP recebe o pedido, gera uma *mensagem de resposta* contendo o objecto pretendido e envia essa mensagem pela sua socket

tempo
↓

HTTP não-persistente (cont.)

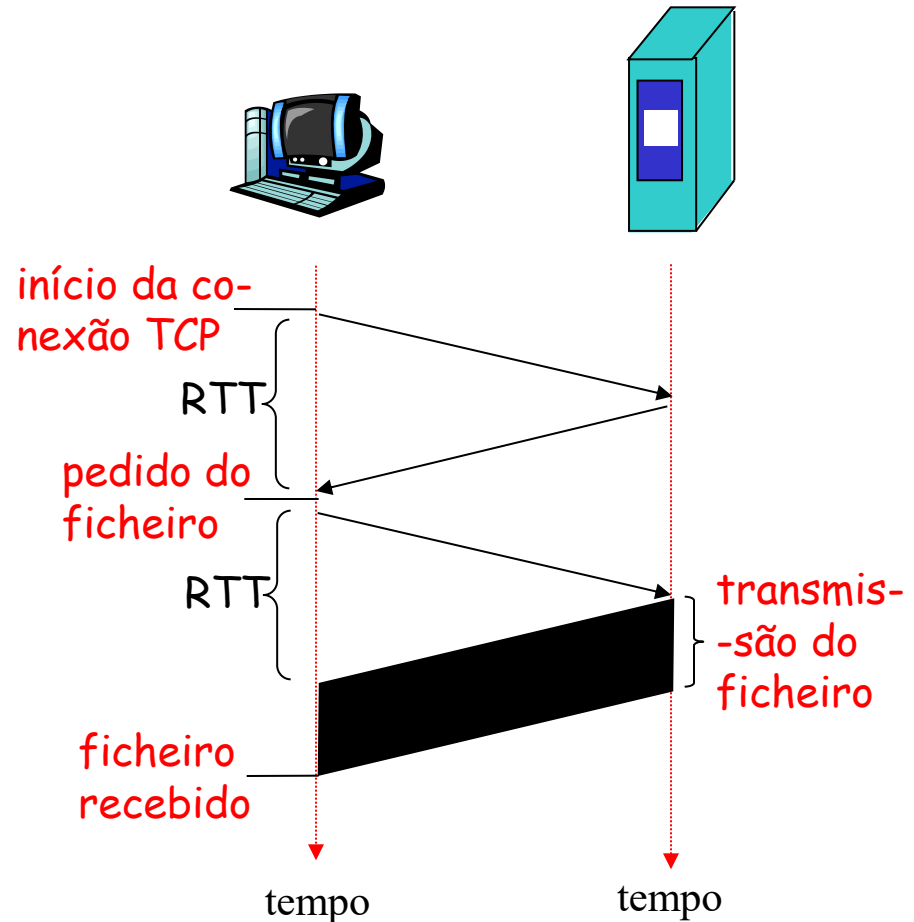


HTTP não-persistente: tempo de resposta

Definição de RTT: tempo de ida e volta de um pequeno pacote entre o cliente e o servidor

Tempo de resposta:

- ❑ um RTT para iniciar a conexão TCP
- ❑ um RTT para o pedido HTTP e os primeiros bytes da resposta
- ❑ transmissão do ficheiro



total = 2RTT + tempo de transmissão do ficheiro

HTTP persistente

Problemas do HTTP não-persistente:

- ❑ exige 2 RTTs por objecto
- ❑ *overhead* no SO para cada conexão TCP estabelecida
- ❑ frequentemente, os browsers usam múltiplas conexões TCP paralelas para obter os diferentes objectos*

HTTP persistente

- ❑ o servidor deixa a conexão aberta para futuras transacções pedido-resposta
- ❑ mensagens HTTP subsequentes são enviadas na conexão já estabelecida

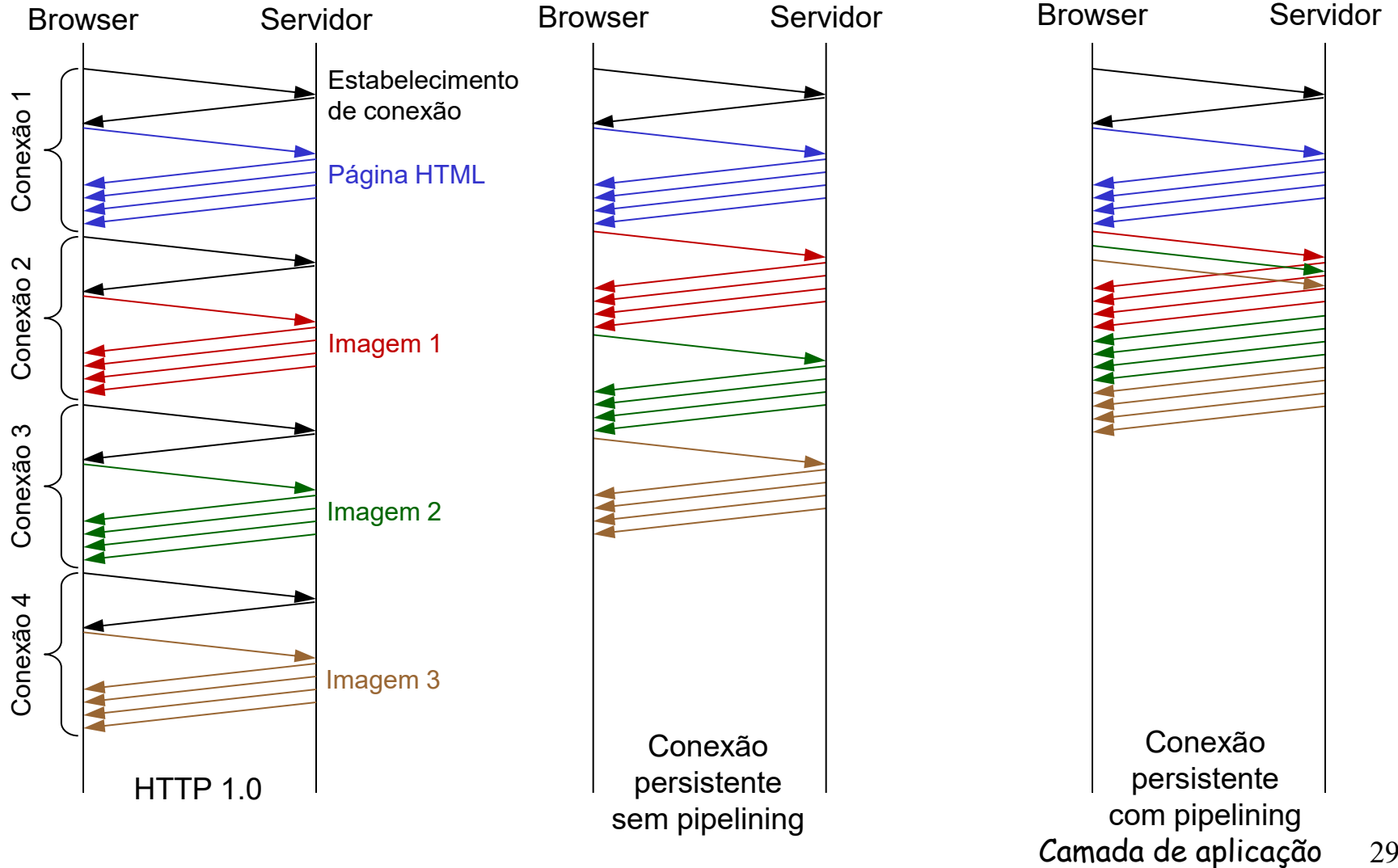
Persistente sem pipelining:

- ❑ cliente faz novo pedido só depois de ter recebido a resposta anterior
- ❑ um RTT por cada objecto referenciado

Persistente com pipelining:

- ❑ padrão no HTTP/1.1
- ❑ o cliente envia o pedido mal encontra a referência ao objecto
- ❑ um só RTT para todos os objectos referenciados

Conexões persistentes e pipelining



Mensagem HTTP de pedido

- ❑ Dois tipos de mensagens HTTP: *pedido* e *resposta*
- ❑ Codificação ASCII (formato humanamente legível)
- ❑ *Pedido HTTP:*

linha de pedido
(comandos GET,
POST, HEAD, ...)

cabeçalhos

```
GET /~rprior/index.html HTTP/1.1
Host: www.dcc.fc.up.pt
User-agent: Mozilla/4.0
Accept: text/html, */*;q=0.9
Accept-language: pt
```

Linha vazia
indica fim da
mensagem

(mudança de linha CRLF extra)

Formato genérico de um pedido HTTP

método		URL		versão	CR	LF
nome do cabeçalho		:	valor		CR	LF
...						
nome do cabeçalho		:	valor		CR	LF
CR	LF					
corpo da mensagem (entidade)						

linha de pedido

linhas de
cabeçalhos

Métodos HTTP

HTTP/1.0

□ GET

- ❖ Pede um recurso ao servidor

□ POST

- ❖ Envia ao servidor dados para processar

□ HEAD

- ❖ pede ao servidor apenas os cabeçalhos

HTTP/1.1

□ GET, POST, HEAD

□ PUT

- ❖ upload de um ficheiro para o URL especificado

□ DELETE

- ❖ apagar o ficheiro especificado no URL

Mensagem HTTP de resposta

linha de estado
(código de estado
e correspondente
descrição)

HTTP/1.1 200 OK

cabeçalhos

Date: Thu, 06 Aug 1998 12:00:15 GMT

Server: Apache/1.3.0 (Unix)

Last-Modified: Mon, 22 Jun 1998 ...

Content-Length: 6821

Content-Type: text/html

linha vazia

dados dados dados dados...

dados (e.g., o
ficheiro HTML
pedido)

necessário para saber
onde termina a mensagem

Códigos de estado nas respostas HTTP

Enviados na primeira linha da resposta do servidor

Alguns códigos:

200 OK

- ❖ sucesso, o objecto pedido vai mais abaixo na mensagem

301 Moved Permanently

- ❖ o objecto foi mudado para outro URL, especificado mais abaixo no cabeçalho "Location:"

400 Bad Request

- ❖ o servidor não entendeu o pedido

404 Not Found

- ❖ o recurso pedido não existe no servidor

505 HTTP Version Not Supported

Experiências com o HTTP

- ❑ À mão, usando o telnet
- ❑ Usando um browser e fazendo a captura dos pacotes enviados e recebidos com o analisador de protocolos *Wireshark*

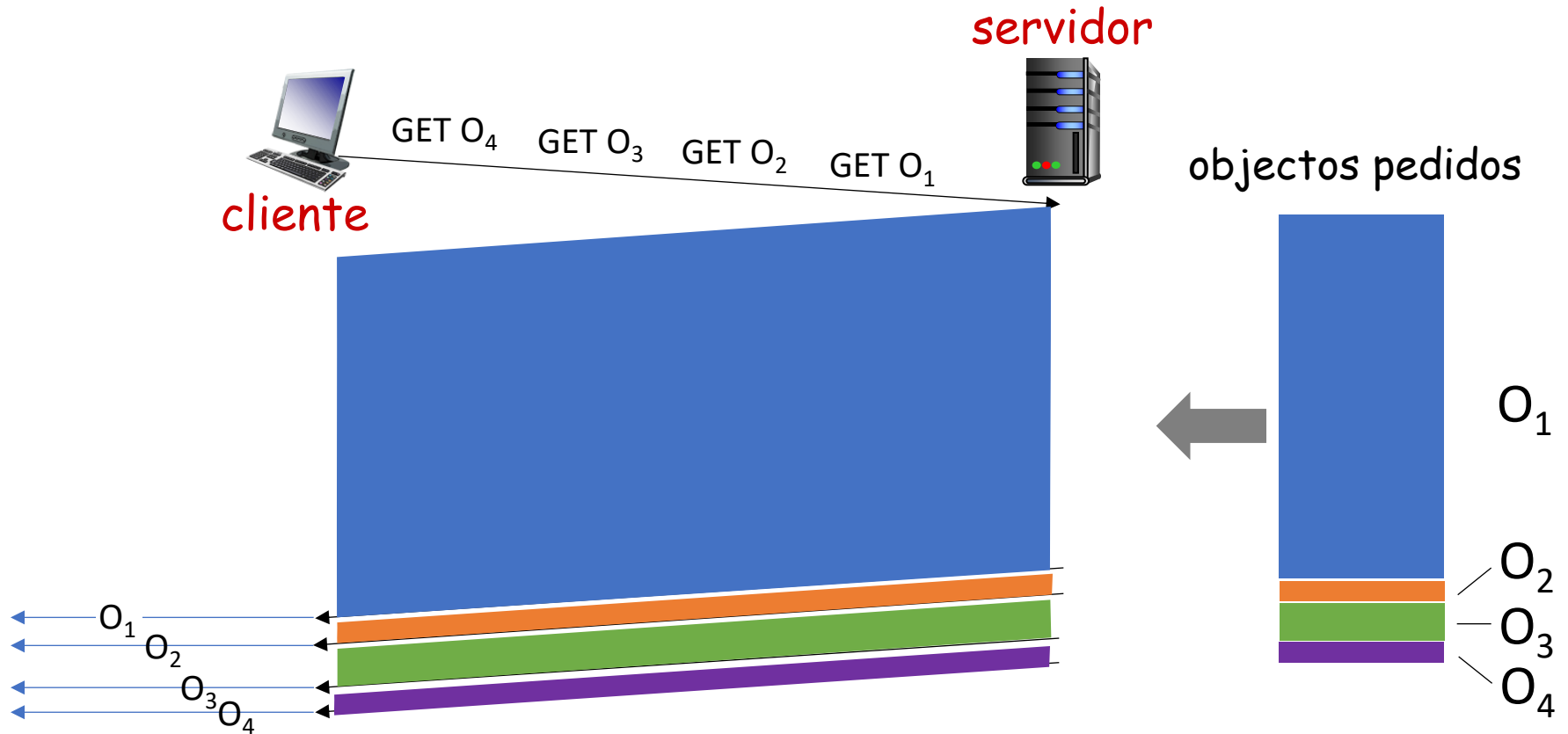
❖ Exemplo

HTTP/2

- ❑ Mesmo com conexões persistentes e *pipelining*, o HTTP 1.1 tem algumas limitações de desempenho
 - ❖ Objetos são entregues pela ordem em que foram pedidos
 - ➔ um objeto grande pode atrasar os que foram pedidos depois: *head-of-line (HOL) blocking*
 - ❖ Recuperação de perdas atrasa todos os objectos
- ❑ HTTP/2 mantém os métodos, códigos de resposta e semântica do HTTP 1.1, mas
 - ❖ Usa codificação binária com multiplexagem de objectos
 - ❖ Suporta priorização de objectos
 - e.g., imagens na parte visível da página são mais urgentes do que as que só aparecem mais à frente
 - ❖ Possibilita *server push* (envio de objectos não pedidos)
 - e.g., para folha de estilo CSS

HTTP 1.1 – Bloqueio HOL

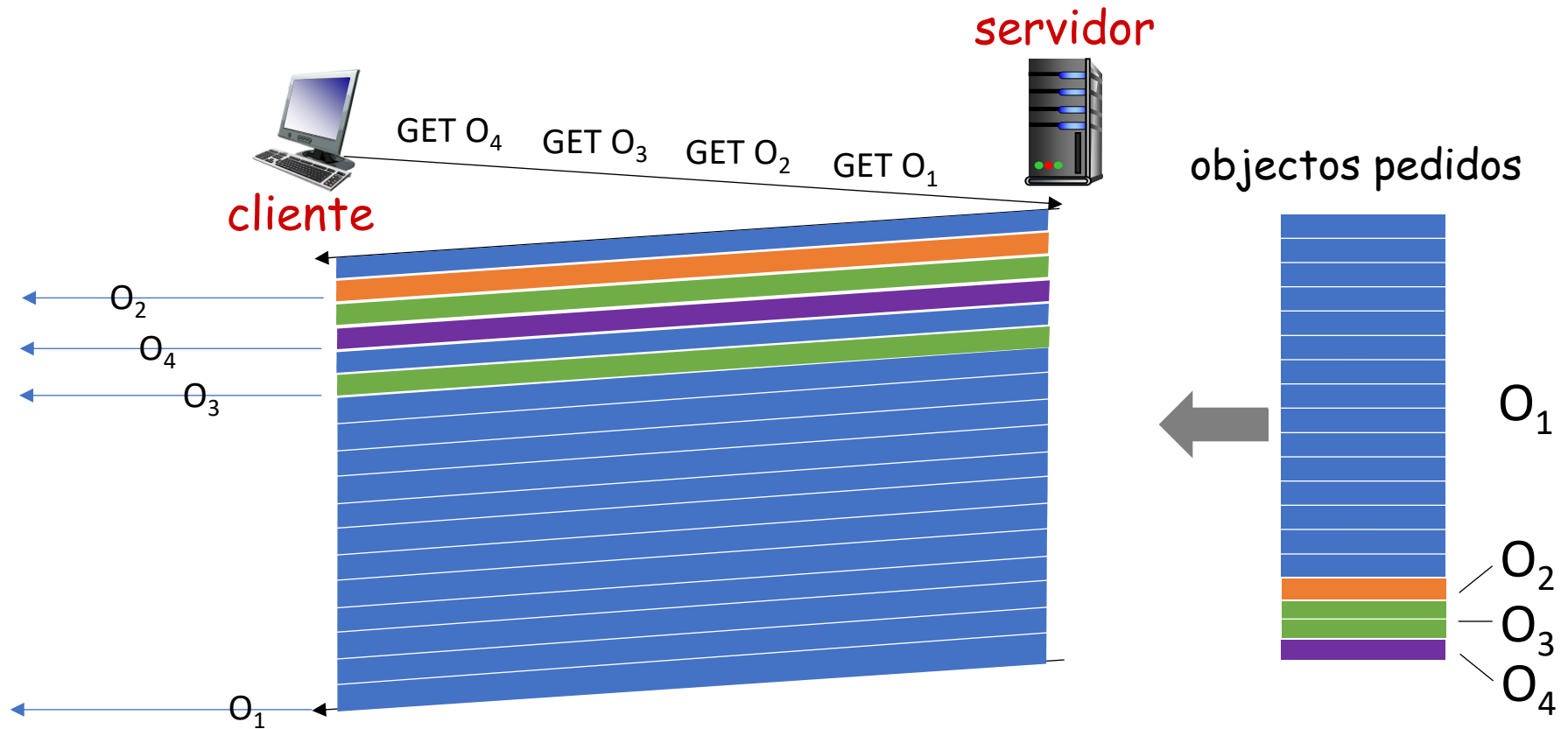
- Transferência de 1 objecto grande seguido de 3 pequenos



Entrega pela ordem pedida: O_2 , O_3 e O_4 têm de esperar por O_1

HTTP/2 Elimina o bloqueio HOL

- Objectos divididos em tramas, transmissão intercalada



O_2 , O_3 e O_4 entregues rapidamente, sem esperar por O_1

HTTP/3

□ HTTP/2 ainda tem algumas limitações

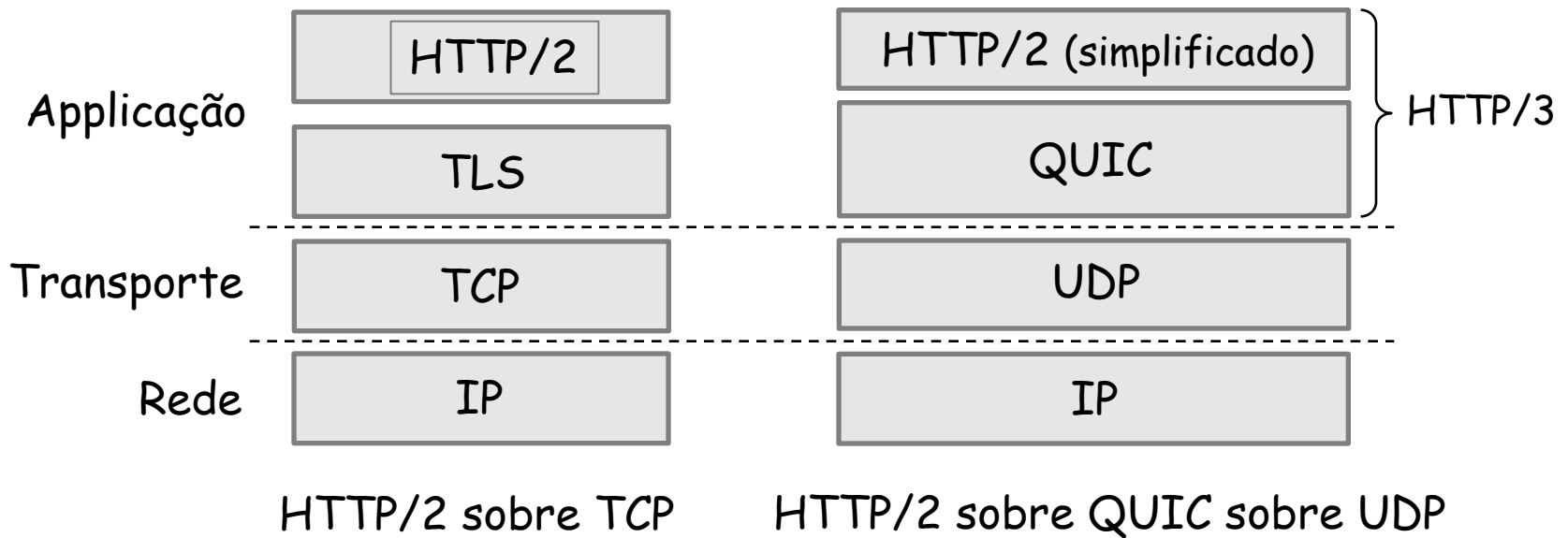
- ❖ Com uma conexão TCP única, recuperação de erros atrasa todos os objectos
 - Incentivo a usar múltiplas conexões paralelas, dificultando o controlo de congestionamento
- ❖ Segurança obriga a um RTT adicional

□ HTTP/3 resolve estes problemas usando um protocolo de transporte diferente

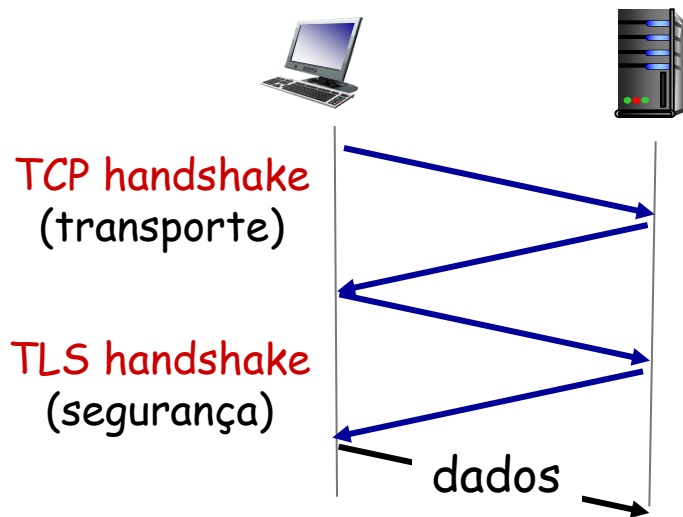
- ❖ QUIC: Quick UDP Internet Connections
 - Implementado na camada de aplicação, funciona sobre UDP
 - Mecanismos próprios para recuperação de erros e controlo de congestionamento

QUIC

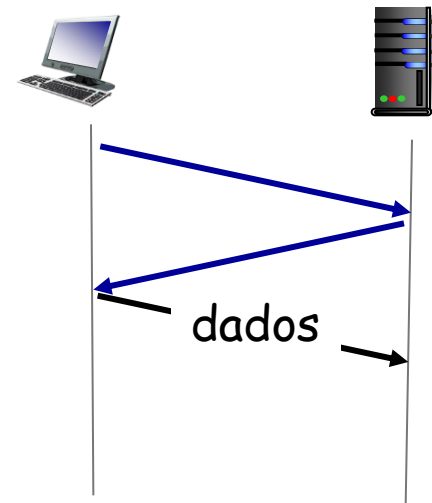
- Implementado na camada de aplicação, sobre UDP



QUIC: Estabelecimento de conexões



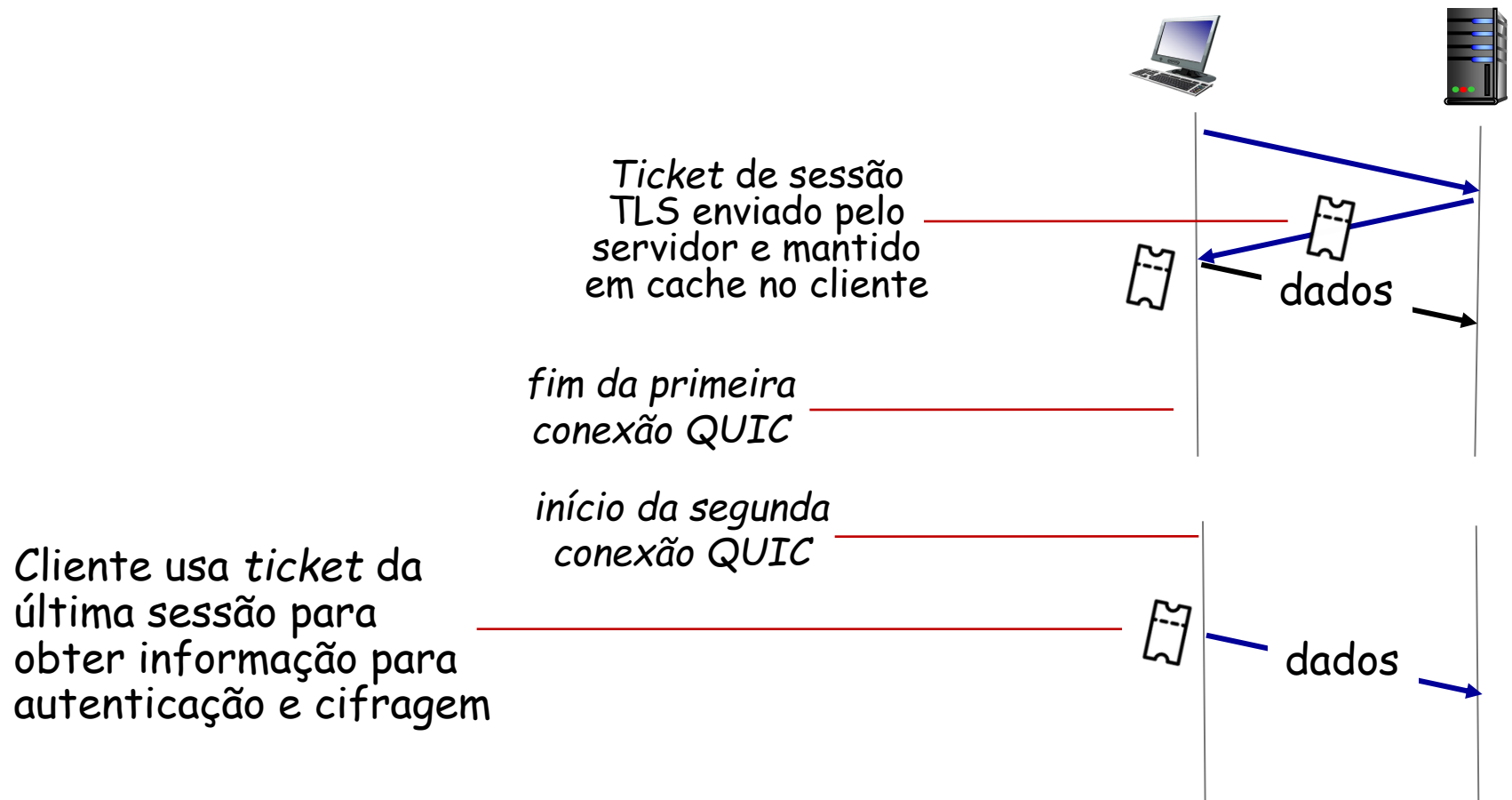
- 2 *handshakes* sequenciais
 - TCP para fiabilidade e controlo de congestionamento
 - TLS para autenticação e cifragem



- 1 *handshake* para tudo

QUIC: Estabelecer conexão com 0 RTT

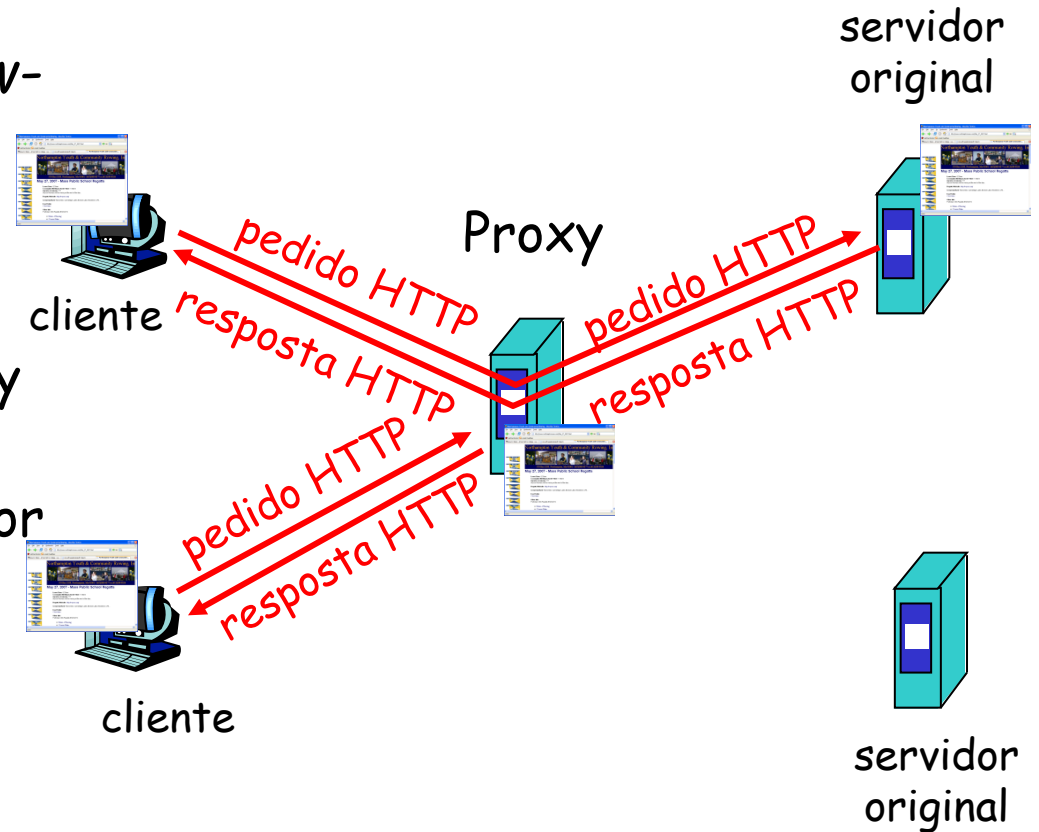
- O RTT inicial pode ser eliminado em conexões subsequentes



Web caches (proxy server)

Objectivo: satisfazer pedidos dos clientes sem recorrer ao servidor original

- utilizador configura *browser* para usar o *proxy**
- o *browser* envia todos os pedidos HTTP ao *proxy*
 - ❖ objecto em cache: proxy devolve-o directamente
 - ❖ senão, pede-o ao servidor original, devolve-o ao browser e guarda cópia em cache



Mais sobre proxies e caching

- ❑ proxy actua como cliente e como servidor
- ❑ normalmente é instalado por uma organização (universidade, empresa, ISP)
- ❑ um proxy pode também registar ou filtrar os pedidos dos utilizadores
- ❑ o próprio browser também faz caching
 - ❖ acessos repetidos à mesma página

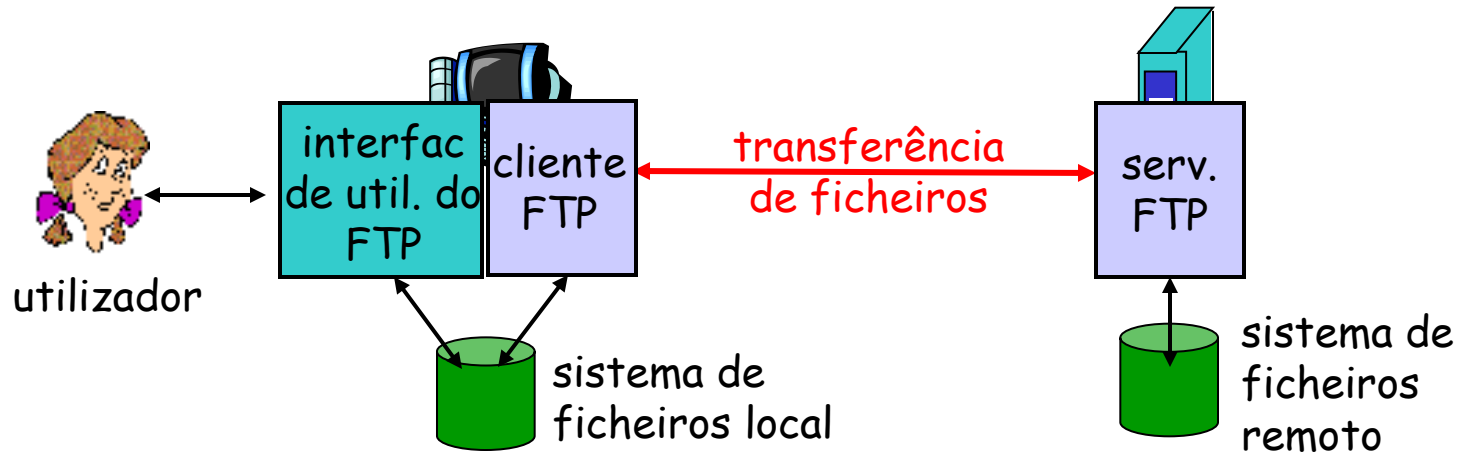
Porquê fazer caching?

- ❑ reduz tempo de resposta aos pedidos
- ❑ reduz tráfego no acesso à Internet
- ❑ abundância de caches permite mesmo aos servidores de baixa capacidade funcionar aceitavelmente

Capítulo 2: Camada de aplicação

- ❑ 2.1 Princípios das aplicações em rede
- ❑ 2.2 Web e HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Correio electrónico
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS
- ❑ 2.6 Aplicações P2P
- ❑ 2.7 Programação com *sockets* — TCP
- ❑ 2.8 Programação com *sockets* — UDP

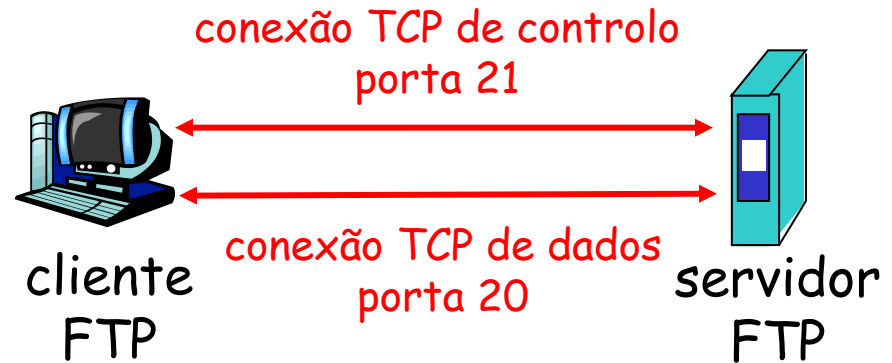
FTP: File Transfer Protocol



- ❑ transferir ficheiros de/para um servidor remoto
- ❑ modelo cliente/servidor
 - ❖ **cliente**: lado que desencadeia a transferência (em qualquer sentido)
 - ❖ **servidor**: máquina remota
- ❑ especificação do ftp: RFC 959
- ❑ porta do servidor ftp: 21

FTP: conexões de controlo e de dados

- ❑ o cliente liga-se ao servidor FTP na porta 21 (usando TCP como protocolo de transporte)
- ❑ autenticação do utilizador na conexão de controlo
- ❑ o cliente navega no sistema de ficheiros remoto enviando comandos na conexão de controlo
- ❑ quando o servidor recebe um comando de transferência de ficheiro abre outra conexão TCP para transferir o ficheiro
- ❑ após a transferência, a segunda conexão é terminada



- ❑ para transferir um novo ficheiro, o servidor abre outra conexão TCP de dados
- ❑ controlo "out of band" (numa conexão separada dos dados)
- ❑ o servidor FTP mantém informação de estado (directório corrente, autenticação)

FTP: comandos e respostas

Alguns comandos:

- ❑ enviados como texto ASCII na conexão de controlo
- ❑ `USER username`
- ❑ `PASS password`
- ❑ `PORT` indica IP e porta do cliente p/conexão de dados
- ❑ `LIST` lista os ficheiros no directório corrente
- ❑ `RETR filename` receber um ficheiro do servidor
- ❑ `STOR filename` enviar um ficheiro para o servidor

Alguns códigos de resposta

- ❑ código de estado e descrição textual (tal como no HTTP)
- ❑ 331 Username OK, password required
- ❑ 125 data connection already open; transfer starting
- ❑ 425 Can't open data connection
- ❑ 452 Error writing file

Exemplo: [captura de sessão com o Wireshark](#)

Capítulo 2: Camada de aplicação

- ❑ 2.1 Princípios das aplicações em rede
- ❑ 2.2 Web e HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Correio electrónico
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS
- ❑ 2.6 Aplicações P2P
- ❑ 2.7 Programação com *sockets* — TCP
- ❑ 2.8 Programação com *sockets* — UDP

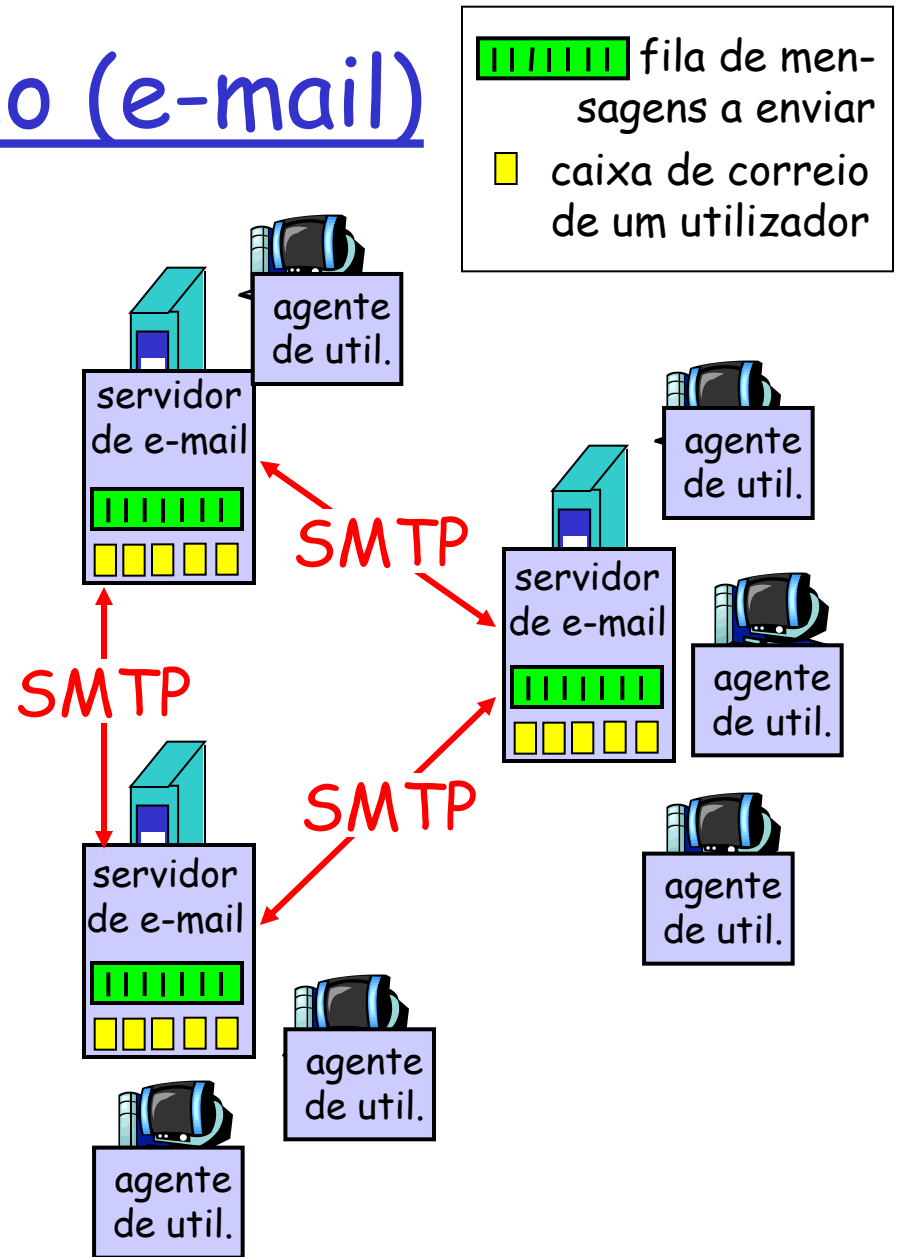
Correio Electrónico (e-mail)

Três componentes:

- ❑ agentes de utilizador
- ❑ servidores de e-mail
- ❑ protocolo SMTP (*simple mail transfer protocol*)

Agente de utilizador

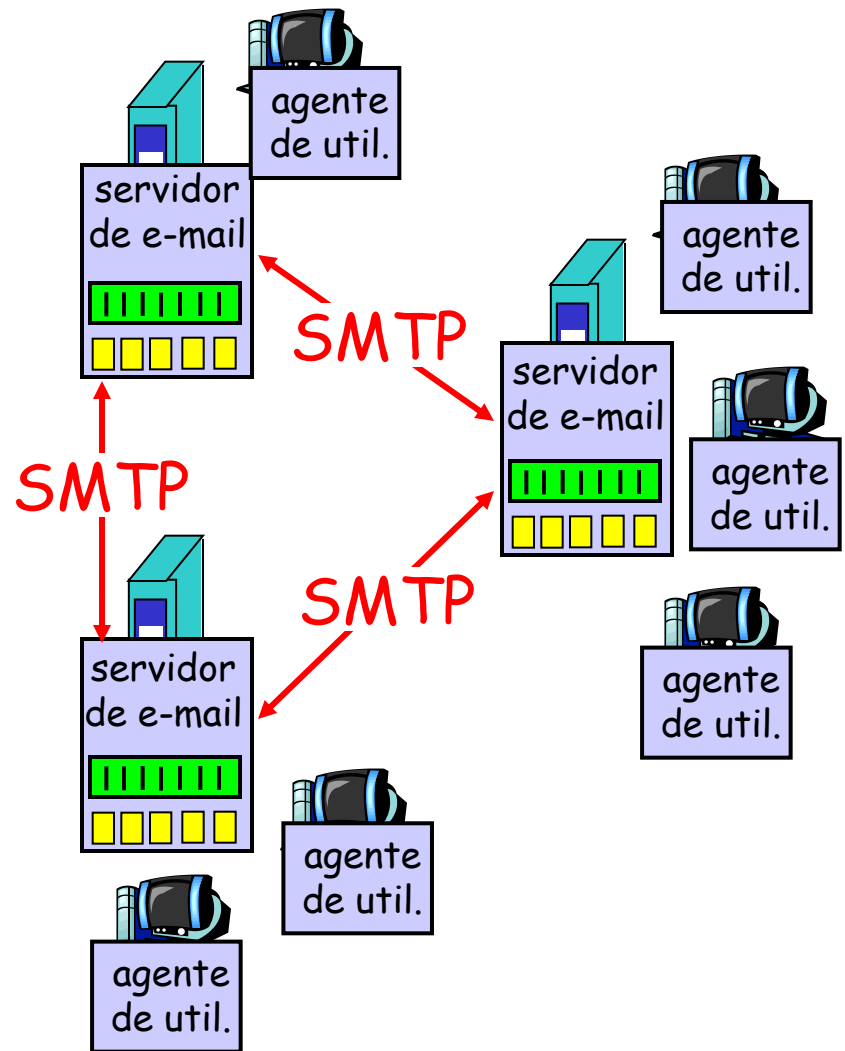
- ❑ leitor/cliente de e-mail
- ❑ e.g., Mozilla Thunderbird, Eudora, Outlook, pine...
- ❑ permite escrever, editar e ler mensagens de e-mail
- ❑ mensagens a enviar e recebidas ficam no servidor*



Correio Electrónico: Servidores

Servidores de E-mail

- ❑ **caixa de correio** contém mensagens recebidas
- ❑ **fila de mensagens** de saída (para serem enviadas)
- ❑ **protocolo SMTP** entre servidores de e-mail para enviar as mensagens
 - ❖ "cliente": servidor que envia a mensagem
 - ❖ "servidor": servidor que recebe a mensagem

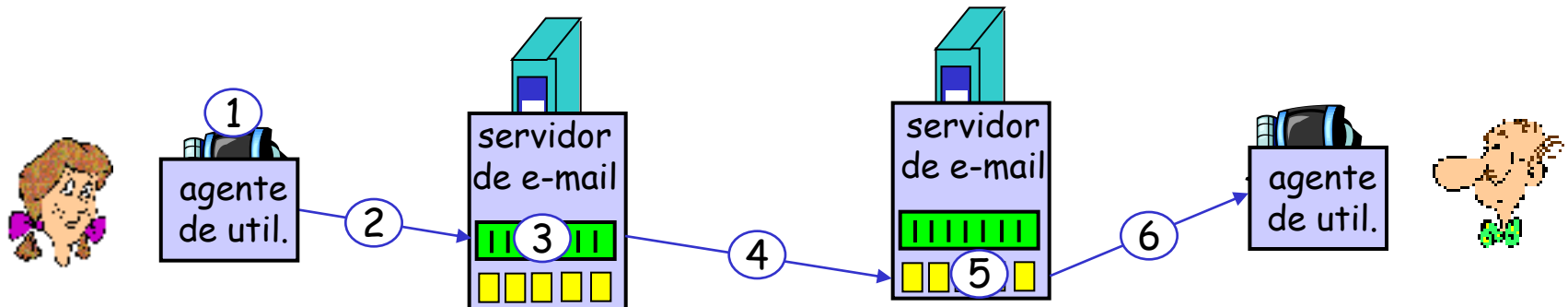


Correio Electrónico: SMTP [RFC 2821]

- ❑ usa o TCP para transferir com fiabilidade a mensagem para o servidor (à escuta na porta 25)
- ❑ transferência directa do servidor do remetente para o do destinatário*
- ❑ três fases para a transferência
 - ❖ *handshaking*
 - ❖ transferência de mensagens
 - ❖ fecho da conexão
- ❑ interacção tipo comando/resposta
 - ❖ **comandos:** texto ASCII
 - ❖ **resposta:** código de estado e respectiva descrição
- ❑ as mensagens têm de estar em ASCII de 7 bits
 - ❖ caracteres acentuados e anexos binários têm de ser codificados

Exemplo: Alice envia mensagem a Bob

- 1) No AU, a Alice escreve o texto da mensagem e o endereço do destinatário `bob@someschool.edu`
- 2) O AU da Alice envia a mensagem para o servidor dela; a mensagem é colocada na fila de saída
- 3) O "lado cliente" do SMTP da Alice abre uma conexão para o servidor de e-mail do Bob
- 4) A mensagem é enviada através dessa conexão TCP
- 5) O servidor de e-mail do Bob põe a mensagem na caixa de correio dele
- 6) O Bob usa o seu AU para ler a mensagem



Exemplo de comunicação SMTP

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

Experimentar o SMTP à mão

Exercícios:

- ❑ experimentar ligar-se à porta 25 do servidor smtp.fc.up.pt usando `telnet smtp.fc.up.pt 25`
- ❑ esperar pela resposta 220 do servidor
- ❑ usar os comandos HELO, MAIL FROM, RCPT TO, DATA e QUIT
 - ❖ isto permite enviar mensagens sem ter de usar um cliente de e-mail (agente de utilizador)
- ❑ experimentar também usar um cliente de e-mail e capturar com o *Wireshark* o envio de uma mensagem
 - ❖ Exemplo

SMTP — algumas notas

- ❑ o SMTP usa conexões persistentes
- ❑ o SMTP exige que a mensagem (cabeçalho e corpo) esteja em formato ASCII de 7 bits
- ❑ **Q:** Como enviar anexos binários?
- ❑ o SMTP usa `CRLF.CRLF` (uma linha com apenas um ponto) para marcar o fim da mensagem

Comparação com o HTTP:

- ❑ HTTP: *pull*
- ❑ SMTP: *push*
- ❑ ambos usam comandos e respostas em ASCII
- ❑ ambos usam códigos de resposta e descrição textual
- ❑ HTTP: cada objecto está encapsulado na sua própria mensagem de resposta
- ❑ SMTP: múltiplos objectos numa única mensagem multiparte

Formato das mensagens de e-mail

SMTP: protocolo para a troca de mensagens de e-mail

RFC 2822: standard para o formato das mensagens:

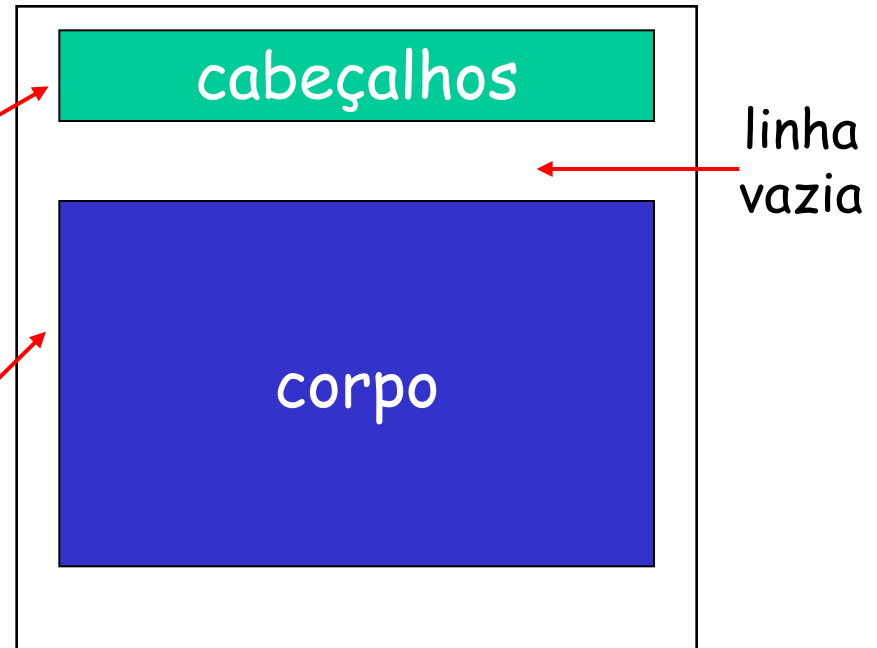
- cabeçalhos, e.g.,

- ❖ To:
- ❖ From:
- ❖ Subject:

diferentes dos comandos SMTP!!!

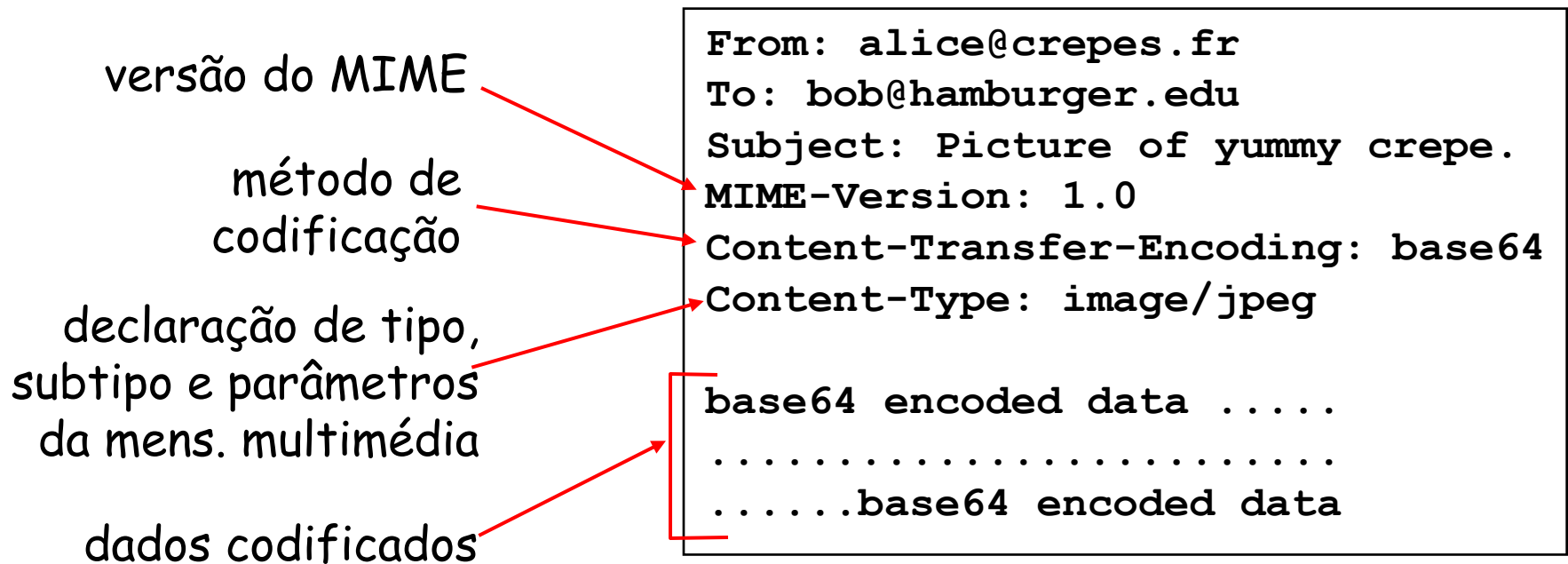
- corpo

- ❖ a "mensagem" (apenas caracteres ASCII)

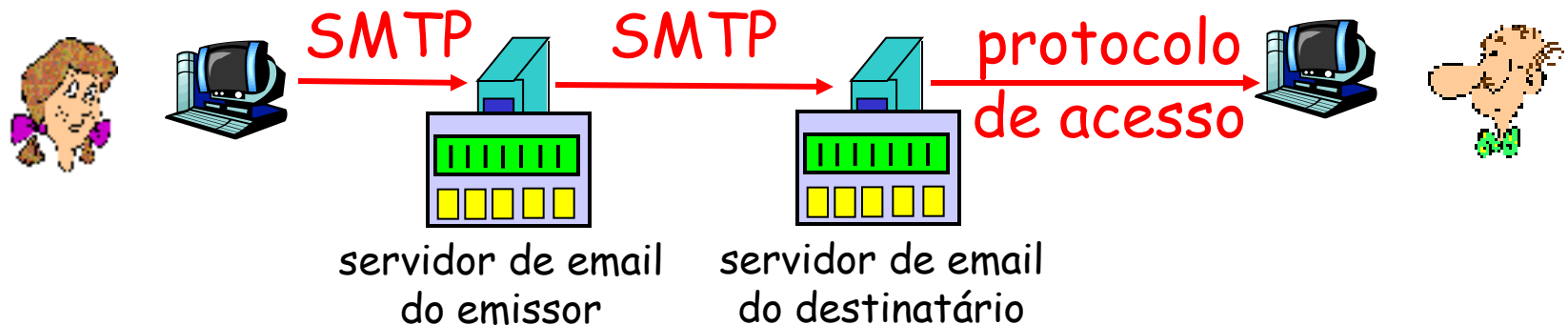


Formato das mensagens: extensões multimédia

- ❑ MIME: extensão multimédia do e-mail, RFC 2045, 2056
- ❑ linhas adicionais no cabeçalho da mensagem declaram o tipo de conteúdo MIME



Protocolos de acesso ao e-mail



- ❑ SMTP: entrega/armazenamento no servidor do destinatário
- ❑ Protocolo de acesso: permite ir buscar e-mail à caixa de correio no servidor
 - ❖ POP: Post Office Protocol [RFC 1939]
 - autorização (agente <--> servidor) e *download*
 - ❖ IMAP: Internet Mail Access Protocol [RFC 1730]
 - mais funcionalidade (mais complexo)
 - manipulação de mensagens armazenadas no servidor
 - ❖ HTTP: Gmail, Proton Mail, etc.

Protocolo POP3

fase de autorização

- ❑ comandos do cliente:
 - ❖ user: indicar username
 - ❖ pass: password
- ❑ respostas do servidor:
 - ❖ +OK
 - ❖ -ERR

fase de transacções

- ❑ list: lista mensagens
- ❑ retr: transfere mensagem
- ❑ dele: apaga mensagem do servidor
- ❑ quit: termina sessão

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on

C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

POP3 vs IMAP

POP3

- ❑ O exemplo anterior usa o modo "transfere e apaga"
- ❑ Bob não pode reler a mensagem se mudar de cliente
- ❑ modo "transfere e deixa ficar" para usar vários clientes
- ❑ POP3 não mantém estado entre sessões

IMAP

- ❑ As mensagens ficam armazenadas no servidor
- ❑ Permite a organização das mensagens em pastas
- ❑ o IMAP mantém estado entre sessões
 - ❖ nomes ds pastas e mapeamento entre mensagens e pastas
- ❑ Mais complexo...

Capítulo 2: Camada de aplicação

- ❑ 2.1 Princípios das aplicações em rede
- ❑ 2.2 Web e HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Correio electrónico
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS
- ❑ 2.6 Aplicações P2P
- ❑ 2.7 Programação com *sockets* — TCP
- ❑ 2.8 Programação com *sockets* — UDP

DNS: Domain Name System

Pessoas: múltiplos identificadores:

- ❖ #CCid, NIF, passaporte

Máquinas na Internet:

- ❖ endereço IP (32 bits), usado no encaminhamento de datagramas
- ❖ nome, usado por humanos (e.g. www.dcc.fc.up.pt)

Q: como se faz o mapeamento entre nomes e endereços IP?

Domain Name System:

- ❑ *base de dados distribuída* implementada como uma hierarquia de *servidores de nomes*
- ❑ as *aplicações*, para comunicar, *resolvem* os nomes (tradução nome/endereço)
 - ❖ serviço fundamental na Internet
 - ❖ implementado como protocolo de aplicação
 - ❖ complexidade nas extremidades da rede

DNS

Serviço DNS

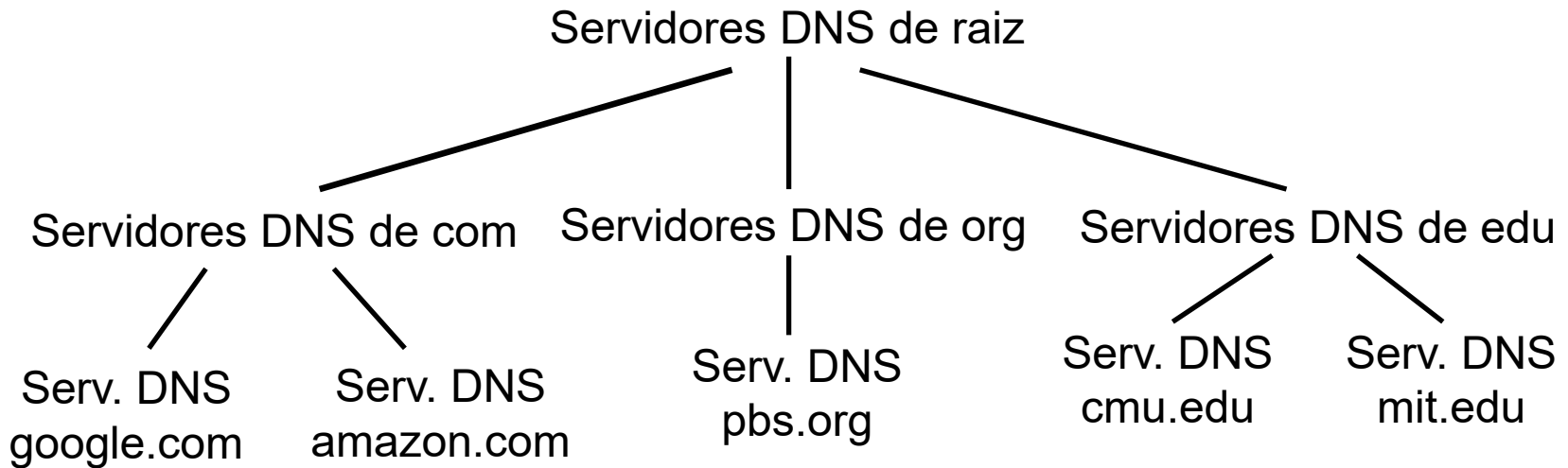
- ❑ tradução de nomes para endereços IP
- ❑ múltiplos nomes (*aliasing*)
 - ❖ Nome canónico, pseudónimos
- ❑ identificação do(s) servidore(s) de e-mail
- ❑ distribuição de carga
 - ❖ replicação de servidores: conjunto de endereços IP para um só nome

Porque não centralizado?

- ❑ seria um ponto de falha
- ❑ volume de tráfego
- ❑ base de dados centralizada distante
- ❑ manutenção seria complexa

Não seria escalável

Base de Dados Distribuída e Hierárquica

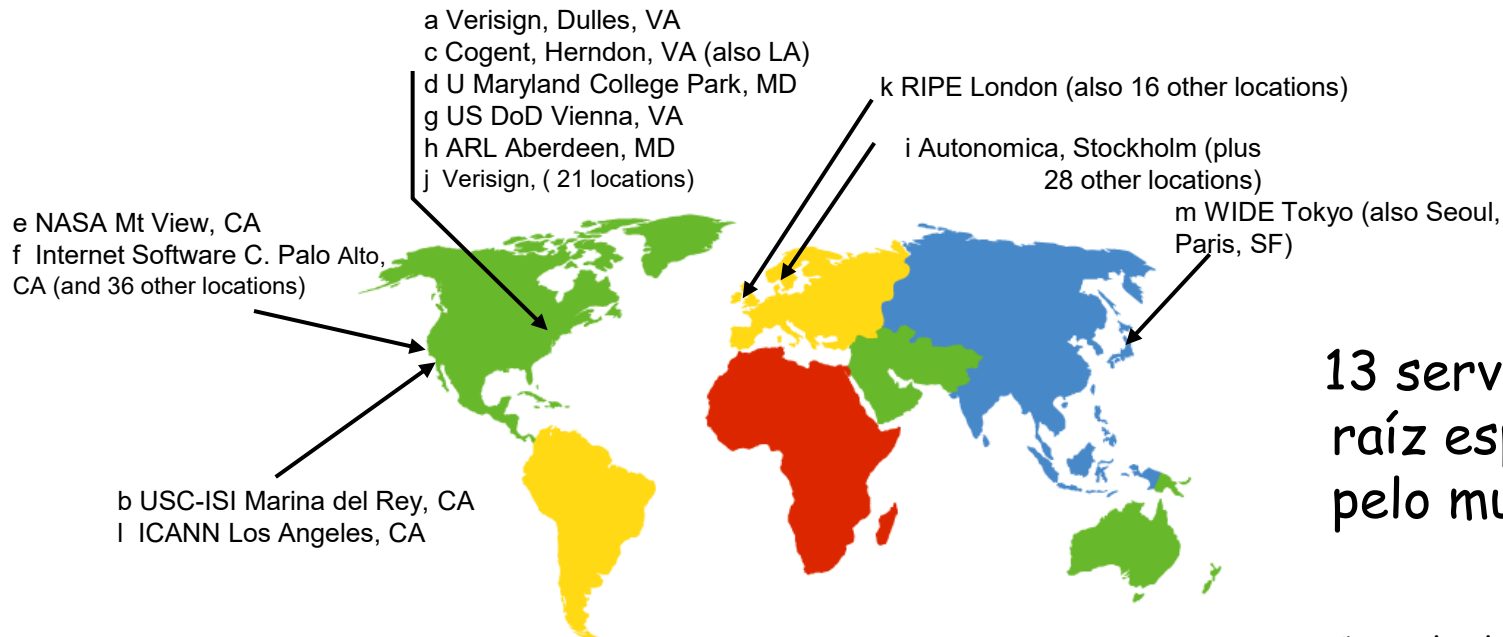


O cliente precisa de obter o IP de `www.amazon.com`. A ideia-base é

- ❑ perguntar a um servidor de raiz, que lhe indica os servidores DNS responsáveis pelo domínio `com`
- ❑ perguntar a um servidor de `com`, que lhe indica os servidores DNS responsáveis pelo domínio `amazon.com`
- ❑ pergunta a um servidor de `amazon.com`, que lhe responde com o endereço IP de `www.amazon.com`

DNS: Servidores de Raiz

- ❑ contactados por um servidor DNS local que não consegue resolver o nome pedido
- ❑ indicam os servidores DNS responsáveis pelo domínio de topo do domínio pedido (e.g., com, pt)
- ❑ para usar o serviço DNS é preciso conhecer o endereço IP (de pelo menos um) deles*



13 servidores de
raiz espalhados
pelo mundo

TLD e Servidores com Autoridade

❑ Servidores de domínio de topo (TLD):

- ❖ responsáveis por com, org, net, edu, etc., e por todos os domínios de topo de país (pt, uk, fr, etc.)
- ❖ a *Network Solutions* mantém os servidores do TLD com
- ❖ a *DNS.PT* mantém os do TLD pt

❑ Servidores DNS com autoridade:

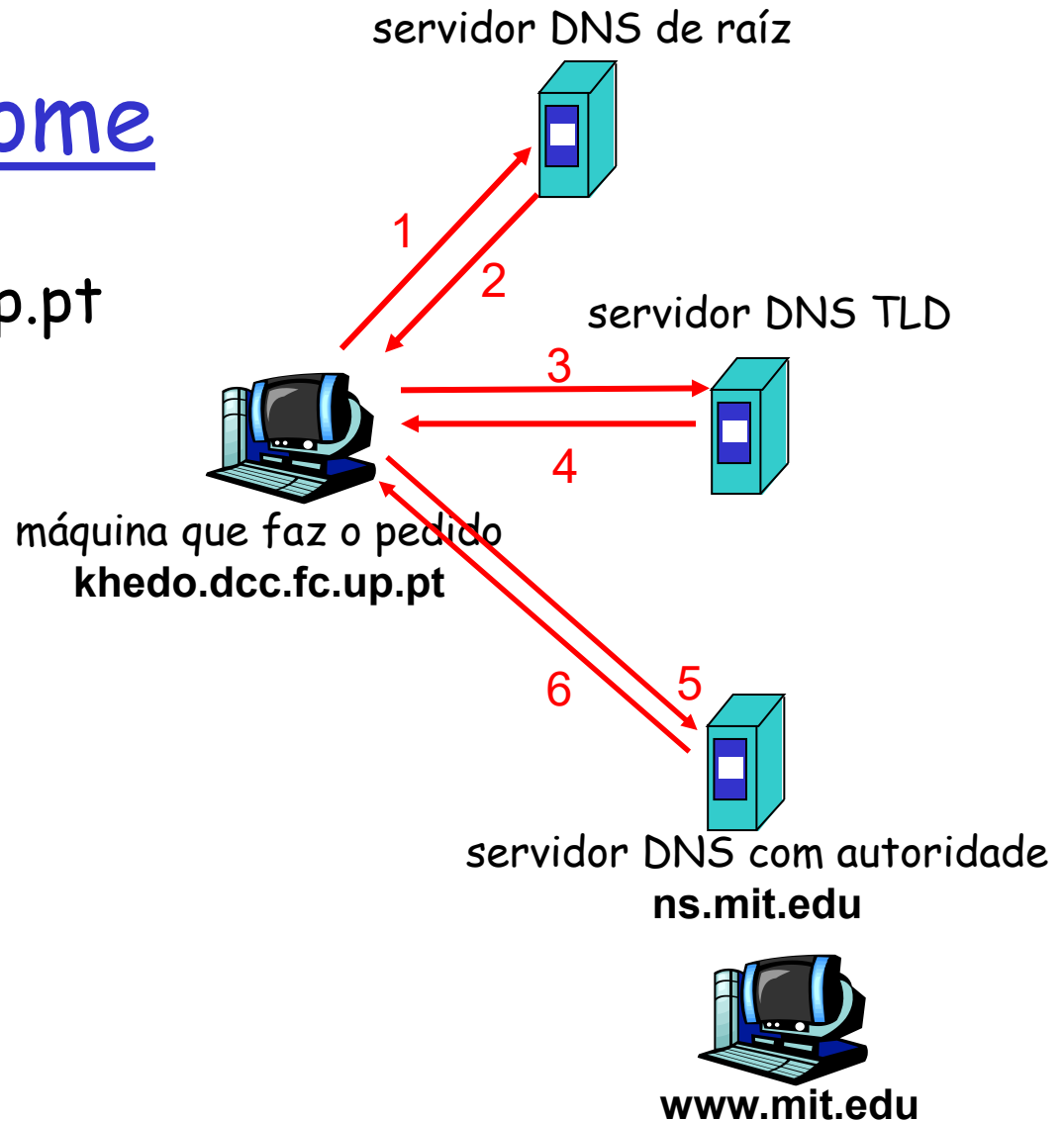
- ❖ servidores DNS de uma organização, detêm autoridade sobre os mapeamentos dos servidores dessa organização (e.g., web, e-mail) para endereços IP
- ❖ podem ser mantidos pela própria organização ou por um ISP em seu nome

Exemplo de resolução de nome

- ❑ Máquina em dcc.fc.up.pt quer o endereço de www.mit.edu

resolução iterativa:

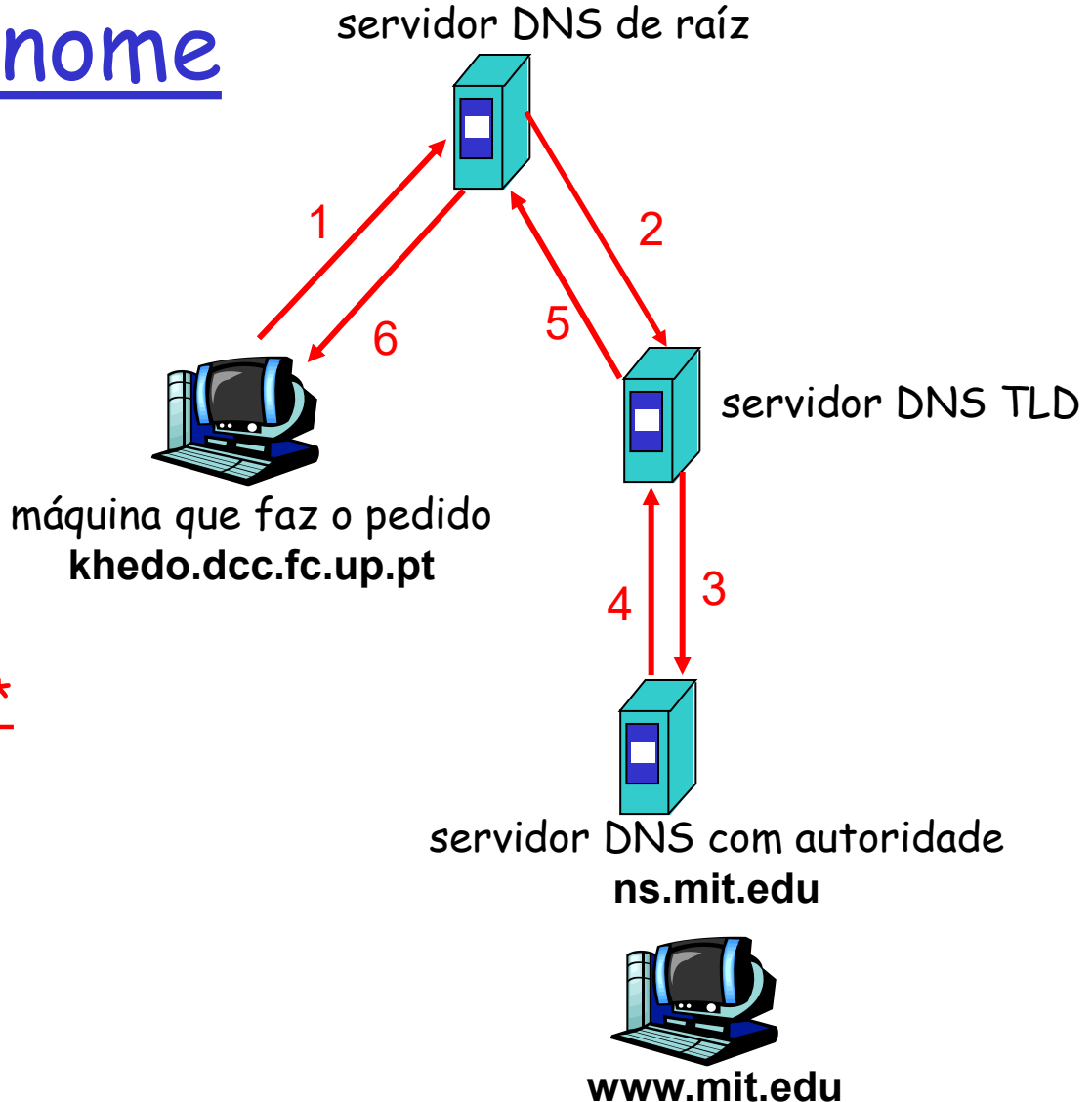
- ❑ o servidor contactado devolve referência a outro(s) servidor(es) a contactar
- ❑ "Não sei resolver este nome, mas pergunta ao servidor X"



Exemplo de resolução de nome

resolução recursiva:

- ❑ o servidor devolve sempre a resposta final (ou um erro)
- ❑ passa o ónus da resolução para o servidor contactado
- ❑ não se usa na prática* sérios problemas de carga, sobretudo nos servidores de raíz...

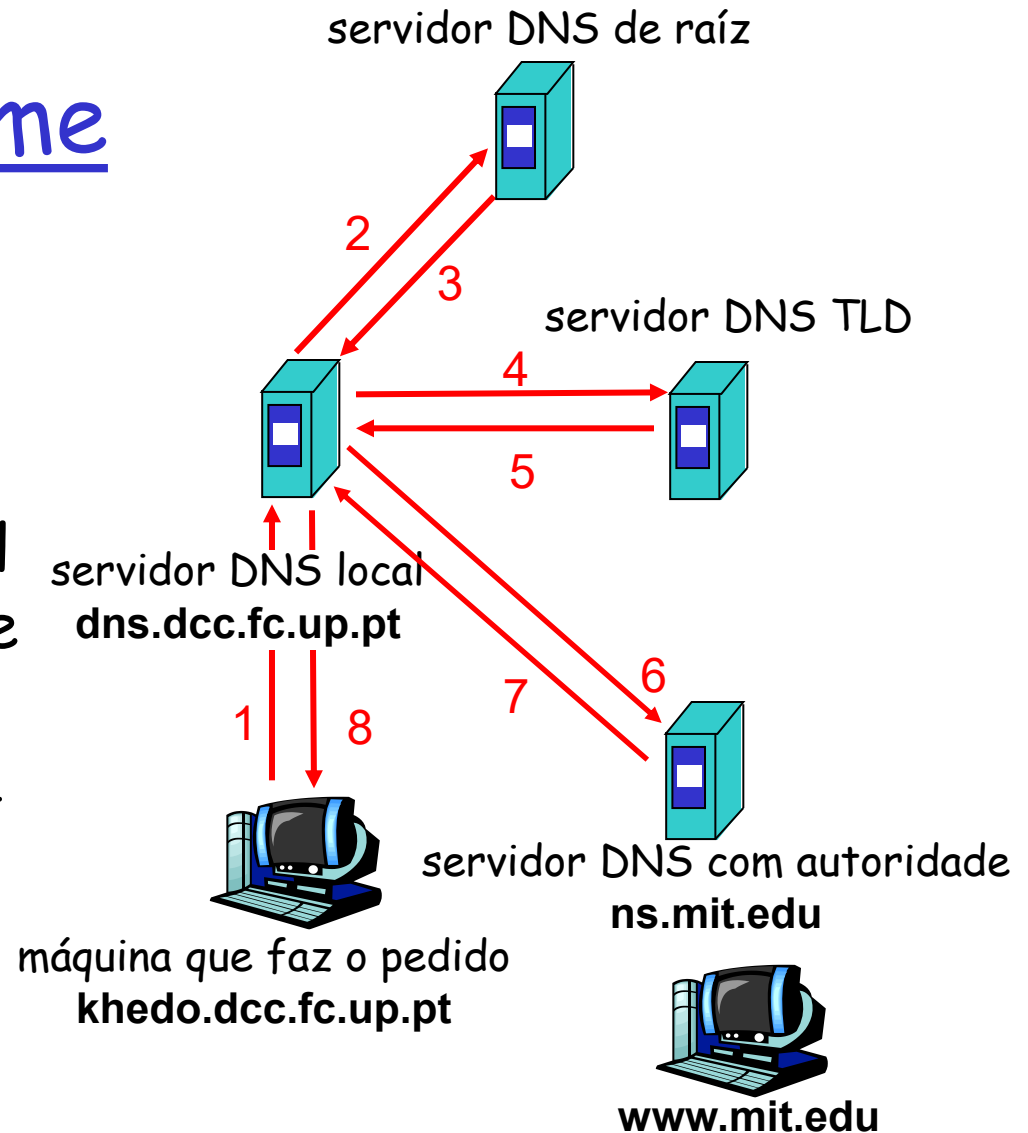


Servidor de Nomes Local

- ❑ não pertence estritamente à hierarquia
- ❑ cada organização (ISP, empresa, universidade) dispõe de um (ou mais)
- ❑ quando uma máquina local precisa de resolver um endereço, envia o pedido ao servidor DNS local
- ❑ servidor DNS local actua como *proxy*
 - ❖ aceita pedidos recursivos das máquinas locais
 - ❖ itera ao longo da hierarquia de servidores até obter a tradução do nome pedido
 - ❖ devolve a resposta obtida à máquina que fez o pedido
 - ❖ faz *caching* das respostas

Caso normal de resolução de nome

- ❑ Terminal faz pedido recursivo ao servidor de DNS local
- ❑ Servidor de DNS local resolve iterativamente o nome
- ❑ Normalmente, um servidor DNS só aceita pedidos recursivos oriundos de máquinas locais



DNS: caches e actualização de registos

- ❑ quando um servidor aprende a resolução de um nome guarda essa resolução em *cache*
 - ❖ entradas na cache são removidas após um certo tempo (*timeout*)
 - ❖ os servidores TLD estão quase sempre em cache nos servidores locais
 - os servidores de raiz raramente são contactados
- ❑ as actualizações aos registos são normalmente feitas pelo administrador da rede
- ❑ existem mecanismos para a actualização dinâmica
 - ❖ RFC 2136
 - ❖ <https://datatracker.ietf.org/wg/dnsind/documents/>

Registos DNS

DNS: BD distribuída de registos de recurso (RR)

Conteúdo de um RR: nome, tipo, valor, ttl

Tipo A

- ❑ nome: nome da máquina
- ❑ valor: endereço IP

Tipo CNAME

- ❑ nome é um pseudónimo para o nome canónico (nome real). E.g., `www.up.pt` é, na realidade, `www.up.pt.cdn.cloudflare.net`
- ❑ valor é o nome canónico

Tipo NS

- ❑ nome: domínio (e.g., `foo.com`)
- ❑ valor: nome de um servidor com autoridade sobre esse domínio

Tipo MX

- ❑ nome: domínio
- ❑ valor: nome do servidor de e-mail desse domínio

Protocolo DNS: mensagens

Protocolo DNS: mensagens de *pergunta* e *resposta*, ambas com o mesmo *formato*, sobre *UDP*

cabeçalho:

- ❑ **identification**: número de 16 bits que permite associar a resposta à pergunta
 - ❖ UDP => sem conexões
- ❑ **flags**:
 - ❖ pergunta ou resposta
 - ❖ recursão desejada
 - ❖ recursão disponível
 - ❖ resposta com autoridade

identification	flags
number of questions	number of answer RRs
number of authority RRs	number of additional RRs
questions (variable number of questions)	
answers (variable number of resource records)	
authority (variable number of resource records)	
additional information (variable number of resource records)	

↑
12 bytes
↓

Protocolo DNS: mensagens

Campos de nome e
tipo da pergunta

RRs de resposta
à pergunta

Registos NS dos servi-
dores com autoridade

Informação adicional
potencialmente útil

identification	flags
number of questions	number of answer RRs
number of authority RRs	number of additional RRs
questions (variable number of questions)	
answers (variable number of resource records)	
authority (variable number of resource records)	
additional information (variable number of resource records)	

↑
12 bytes
↓

As mensagens DNS utilizam um **formato binário**

Protocolo DNS: exemplo de resposta

```
[rprior@localhost ~]$ dig mit.edu
```

```
; <<>> DiG 9.5.0-P1 <<>> mit.edu
```

```
;; global options: printcmd
```

```
;; Got answer:
```

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 28372
```

```
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 3, ADDITIONAL: 3
```

```
;; QUESTION SECTION:
```

```
mit.edu.                IN      A
```

```
;; ANSWER SECTION:
```

```
mit.edu.                60      IN      A      18.7.22.69
```

```
;; AUTHORITY SECTION:
```

```
mit.edu.                11809   IN      NS      STRAWB.mit.edu.
```

```
mit.edu.                11809   IN      NS      BITSY.mit.edu.
```

```
mit.edu.                11809   IN      NS      W20NS.mit.edu.
```

```
;; ADDITIONAL SECTION:
```

```
BITSY.mit.edu.          363     IN      A      18.72.0.3
```

```
W20NS.mit.edu.          3667    IN      A      18.70.0.160
```

```
STRAWB.mit.edu.         3667    IN      A      18.71.0.151
```

Inserção de registos no DNS

- ❑ Exemplo: nova empresa chamada *Network Utopia*
- ❑ Registrar o nome networkutopia.com num agente de registo (DNS *registrar*) (e.g., *Network Solutions*)
 - ❖ fornecer nomes e endereços IP do(s) servidor(es) com autoridade sobre o domínio
 - ❖ o agente de registo insere dois RRs no servidor TLD:

```
networkutopia.com NS dns1.networkutopia.com  
dns1.networkutopia.com A 212.212.212.1
```

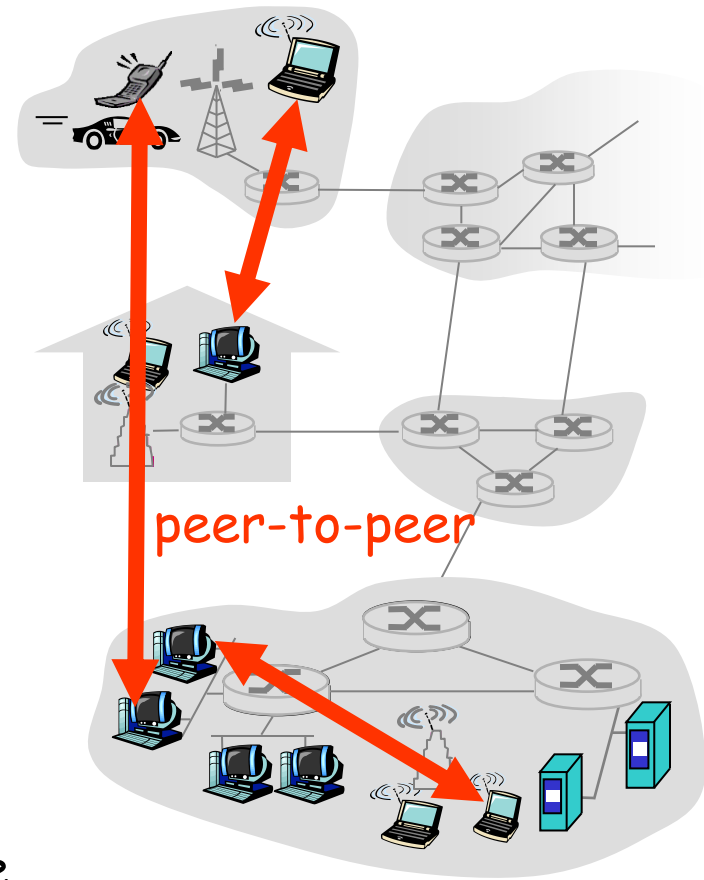
- ❑ No servidor com autoridade, a empresa cria os registos para os nomes dentro do novo domínio
 - ❖ um registo tipo A para www.networkutopia.com, um tipo MX para networkutopia.com, etc.

Capítulo 2: Camada de aplicação

- ❑ 2.1 Princípios das aplicações em rede
- ❑ 2.2 Web e HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Correio electrónico
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS
- ❑ 2.6 Aplicações P2P
- ❑ 2.7 Programação com *sockets* — TCP
- ❑ 2.8 Programação com *sockets* — UDP

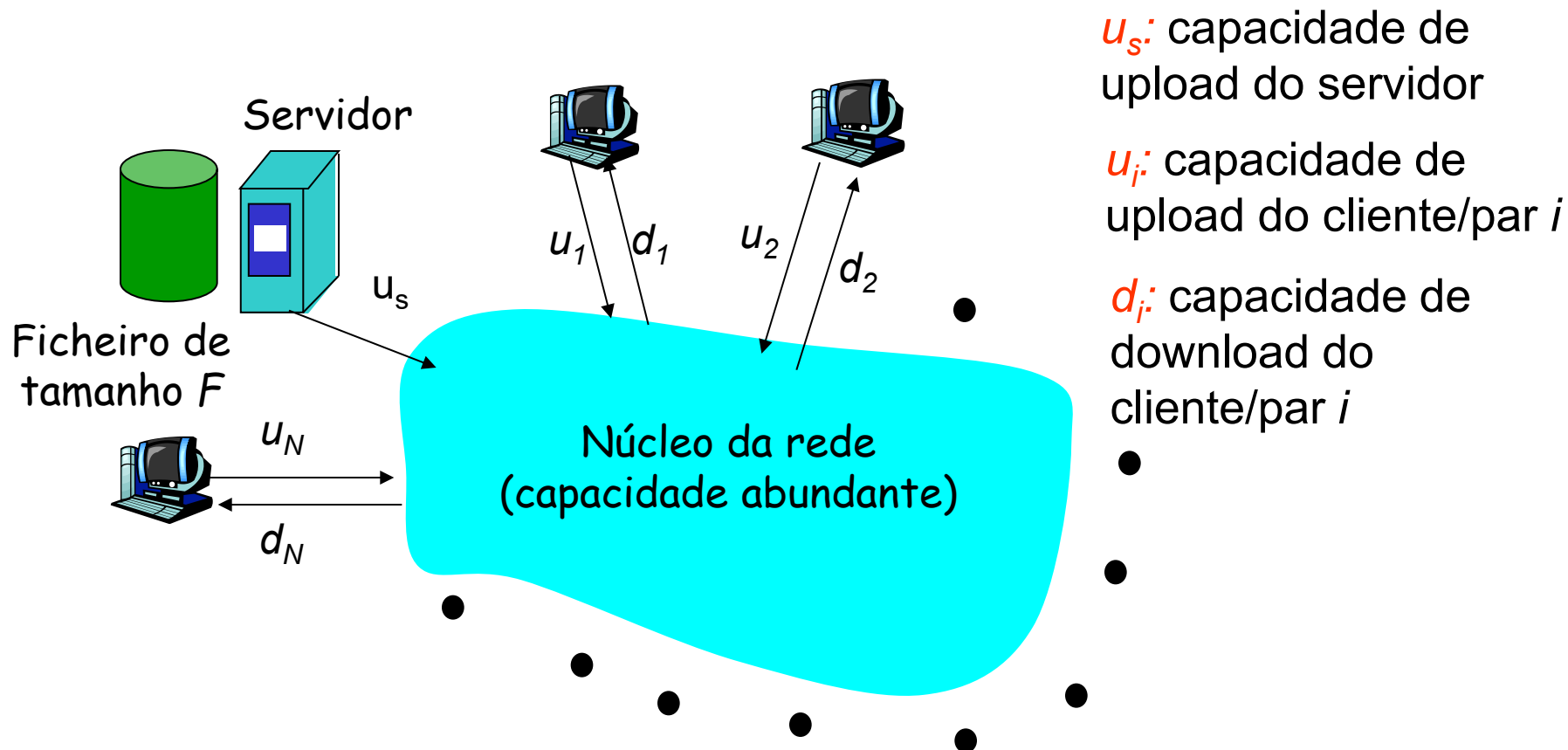
Arquitectura P2P pura

- ❑ sem servidores dedicados
- ❑ terminais comunicam directamente entre si
- ❑ os *peers* podem não estar sempre ligados e mudar de endereço IP
- ❑ **muito escalável**
 - ❖ aumento simultâneo na procura e na oferta
- ❑ **difícil de gerir**
 - ❖ nós ligados intermitentemente
 - ❖ endereços mudam



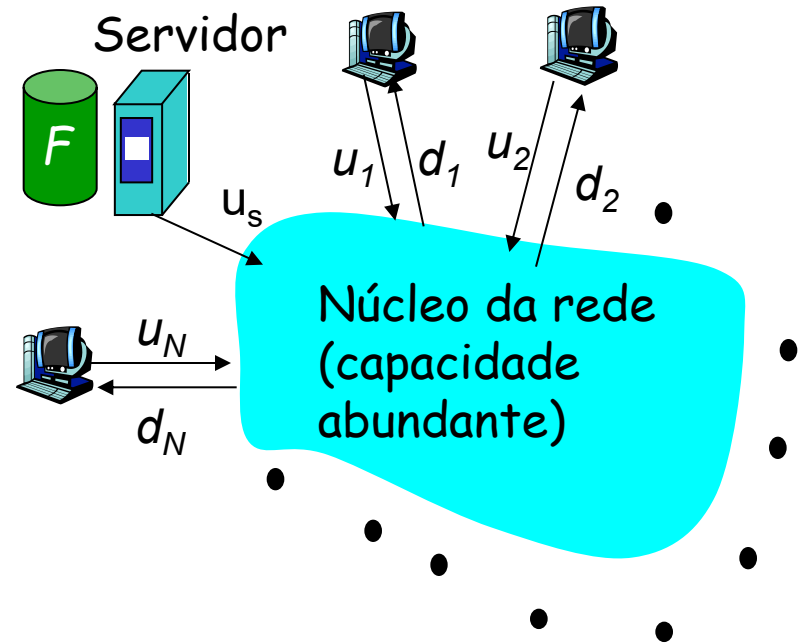
Comparação de arquitecturas: Cliente-Servidor vs. Peer-to-Peer

Questão : Quanto demora a distribuição inicial de um ficheiro de um computador para N outros?



Cliente-servidor: tempo de distribuição

- o servidor envia N cópias do ficheiro:
 - ❖ demora, pelo menos, NF/u_s
- o cliente i demora, pelo menos, F/d_i a fazer o download



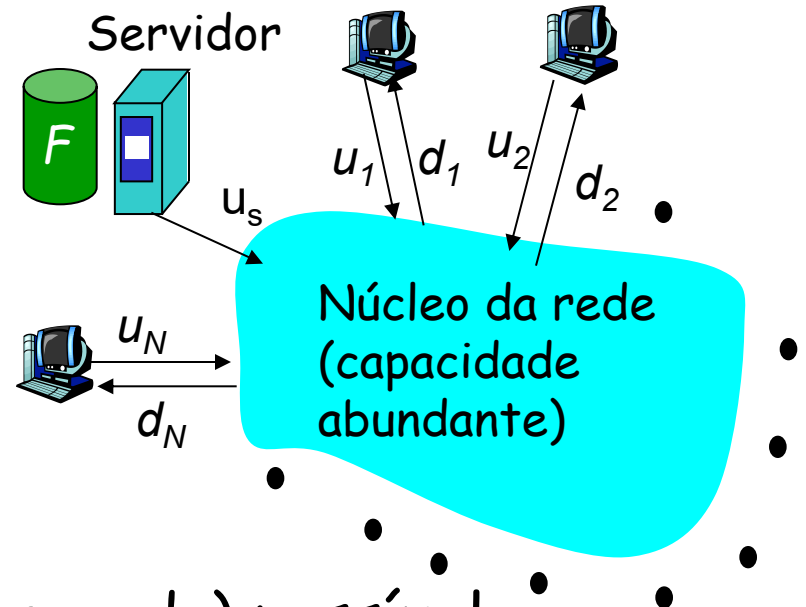
Tempo que demora a distribuir F a N clientes no modelo cliente/servidor

$$d_{cs} = \max \left\{ NF/u_s, F/\min_i(d_i) \right\}$$

Aumenta linearmente com N
(para N grande)

P2P: tempo de distribuição

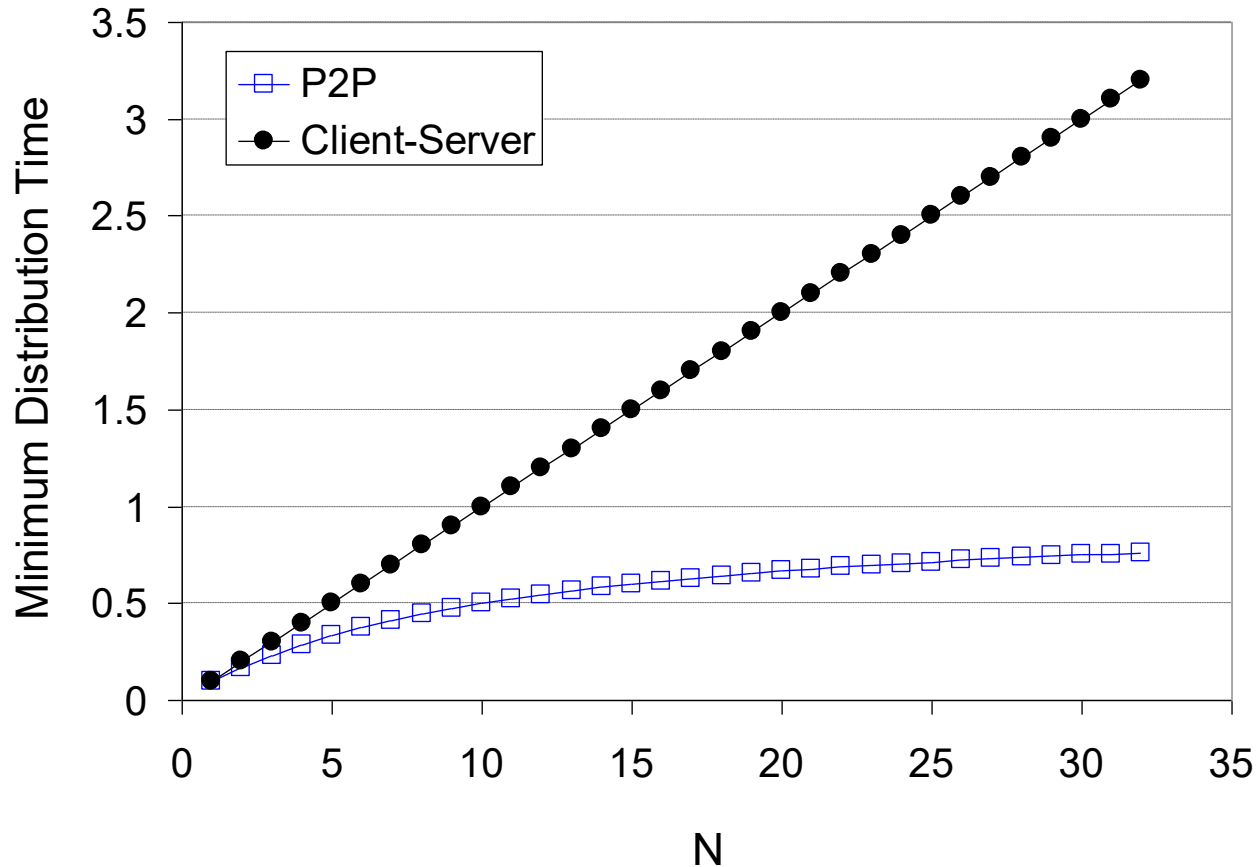
- ❑ o servidor demora F/u_s a enviar ficheiro completo
- ❑ o cliente i demora F/d_i a fazer o download
- ❑ têm de ser transferidos NF bits (agregado)
 - ❑ a maior taxa de envio (agregada) possível é $u_s + \sum_{i=1..N} u_i$



$$d_{P2P} = \max \left\{ F/u_s, F/\min_i(d_i), NF/(u_s + \sum_{i=1}^N u_i) \right\}$$

Tanto o numerador como o denominador crescem com N

Comparação de arquiteturas: Cliente-Servidor vs. Peer-to-Peer

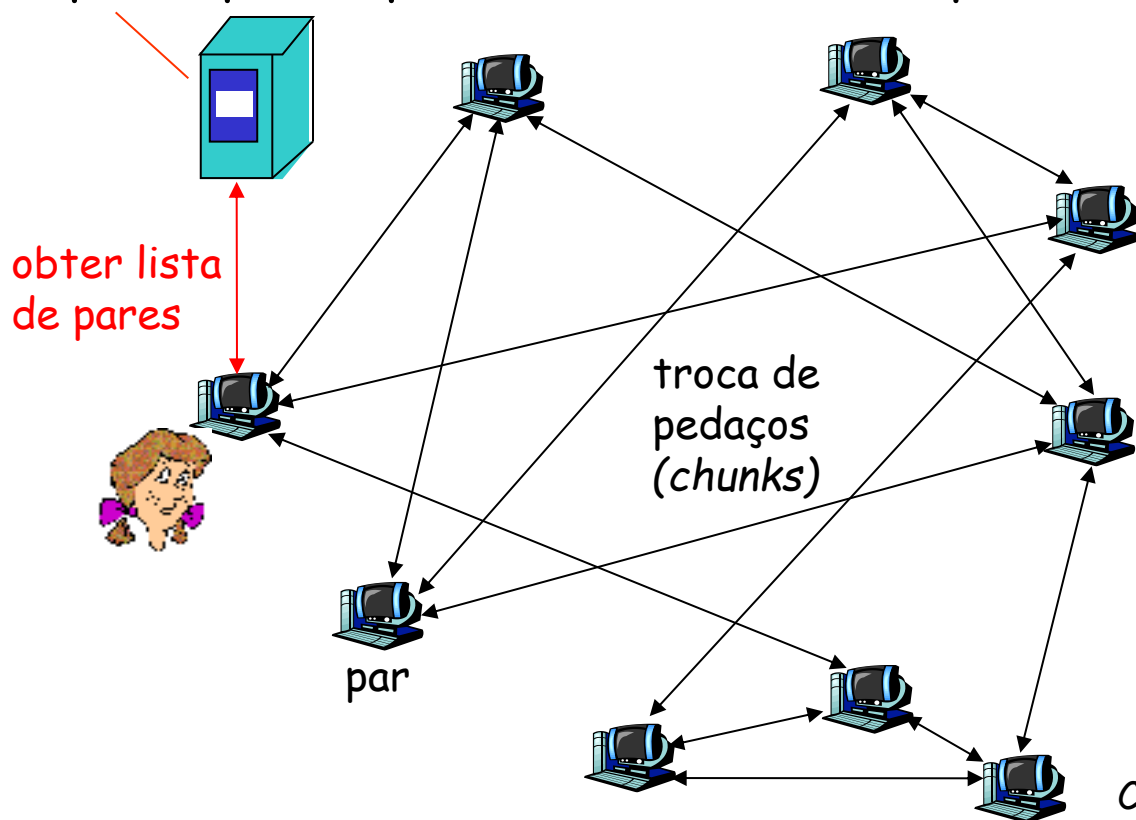


Partilha de ficheiros P2P: BitTorrent

- ❑ Ficheiros divididos em pedaços (normalmente de 256kB)
- ❑ Pares no torrent enviam e recebem pedaços

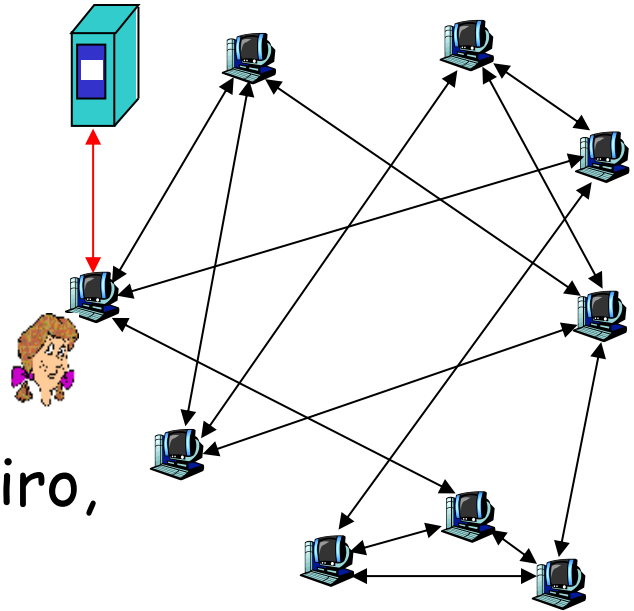
tracker: mantém informação sobre os pares participantes

torrent: grupo de pares a participar na troca de pedaços de um ficheiro



BitTorrent (1)

- ❑ um par que se junta ao *torrent*:
 - ❖ ainda não tem pedaços do ficheiro, mas vai obtê-los
 - ❖ regista-se no *tracker* para obter a lista de pares e liga-se a um subconjunto desses pares ("vizinhos")
- ❑ enquanto faz o *download*, vai fazendo *upload* dos pedaços que já recebeu para outros pares
- ❑ podem variar os pares com quem se trocam pedaços
- ❑ podem entrar novos pares e sair antigos (rotatividade)
- ❑ quando um par tem o ficheiro completo pode sair do *torrent* (egoísta) ou manter-se nele (altruísta)



BitTorrent (2)

Obtenção de pedaços: rarest first

- num dado instante, pares diferentes têm partes diferentes do ficheiro
- periodicamente, cada par pergunta a cada vizinho que pedaços tem
- pede aos vizinhos os pedaços que ainda não tem, **começando pelos mais raros**

Envio de pedaços: tit-for-tat

- um nó envia pedaços aos seus quatro vizinhos que lhe estão a enviar mais
 - ❖ reavaliação a cada 10 s
- de 30 em 30 s: escolhe outro nó aleatoriamente e envia-lhe pedaços
 - ❖ este novo par pode juntar-se ao "top 4"
- quem envia mais, recebe mais...

Capítulo 2: Camada de aplicação

- ❑ 2.1 Princípios das aplicações em rede
- ❑ 2.2 Web e HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Correio electrónico
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS
- ❑ 2.6 Aplicações P2P
- ❑ 2.7 Programação com sockets — TCP
- ❑ 2.8 Programação com sockets — UDP

Programação com sockets

Objectivo: aprender como se constrói uma aplicação cliente/servidor que comunica através de sockets

API de sockets

- ❑ introduzida em 1981 com o BSD 4.1
- ❑ criadas, usadas e libertadas explicitamente
- ❑ paradigma cliente/servidor
- ❑ dois tipos de serviço de transporte fornecidos:
 - ❖ não-fiável de datagramas
 - ❖ fiável de sequência (*stream*) de bytes

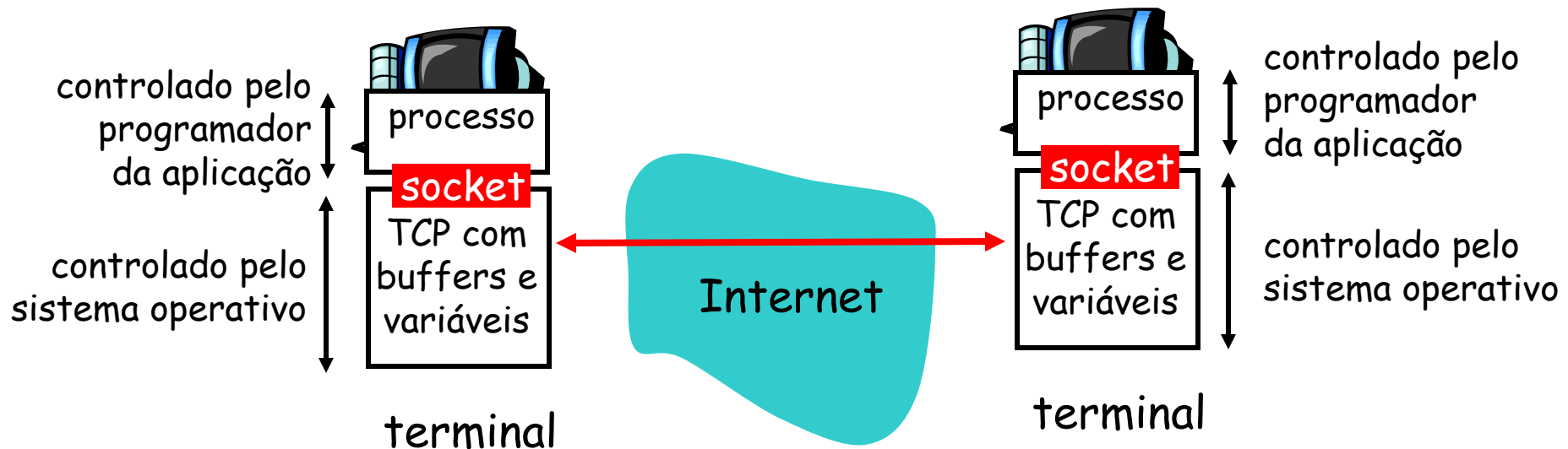
socket

interface *local*,
criada pela aplicação,
controlada pelo SO (uma
"porta") através da qual
os processos (aplicações)
podem *enviar e receber*
mensagens para/de outros
processos (aplicações)

Programação com sockets: TCP

Socket: uma "porta" entre o processo (aplicação) e um protocolo de transporte extremo-a-extremo (TCP ou UDP)

Serviço TCP: transferência fiável de uma **sequência de bytes** de um processo para outro



Programação com sockets: TCP

Servidor espera ser contactado

- processo servidor tem de estar a correr
- servidor à escuta numa socket TCP, à espera de ser contactado pelo cliente

Cliente contacta servidor

- cria uma socket TCP
- especifica o endereço IP e a porta do processo servidor
- estabelece uma conexão TCP para o servidor

- quando contactado pelo cliente, o servidor **cria uma nova socket TCP** para comunicar com o cliente
 - ❖ pode falar simultaneamente com múltiplos clientes
 - ❖ IP e porta de origem permitem distinguir clientes

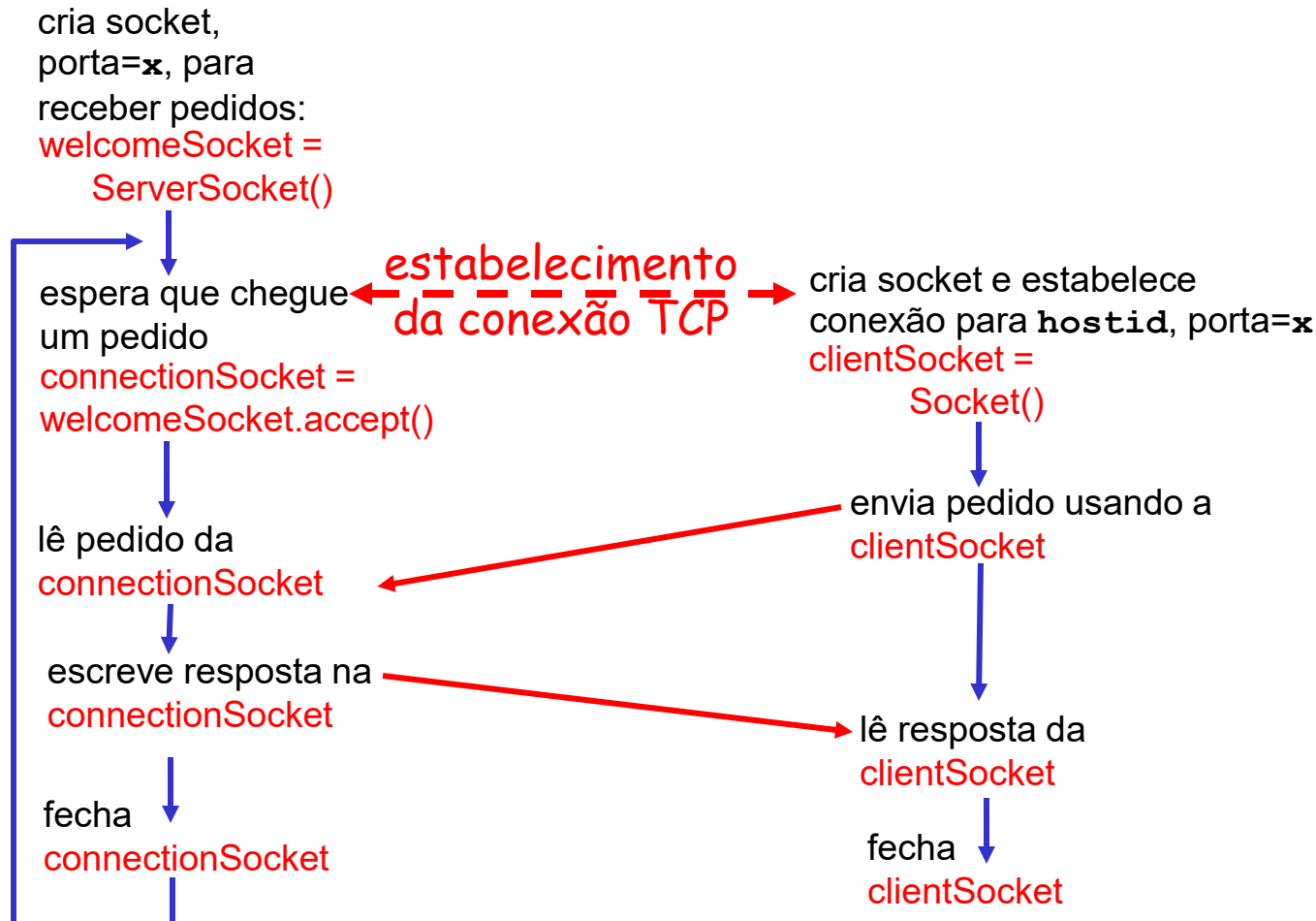
do ponto de vista da aplicação

TCP fornece serviço de transferência de fiável e ordenada de bytes entre cliente e servidor

Sockets no cliente e no servidor: TCP

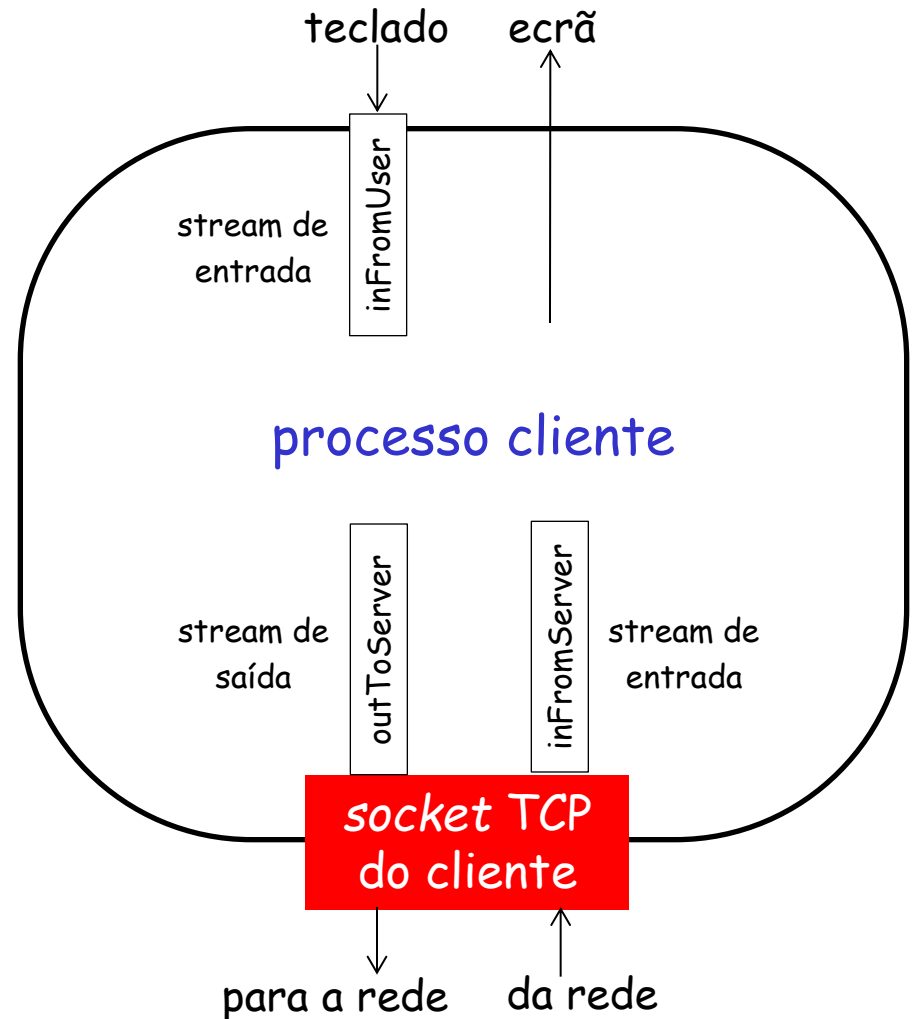
Servidor (a correr em `hostid`)

Cliente



Alguma terminologia

- Uma **stream** é uma sequência de caracteres que entra ou sai de um processo.
- Uma **stream de entrada** está associada a uma fonte de entrada para o processo (teclado ou socket).
- Uma **stream de saída** está associada a uma fonte de saída (ecrã ou socket).



Programação com sockets: TCP

Exemplo de aplicação cliente-servidor:

- 1) O Cliente lê linha da entrada-padrão (`stream inFromUser`) e envia-a ao servidor através da `socket` (`stream outToServer`)
- 2) O Servidor lê linha da `socket`
- 3) O Servidor converte a linha para maiúsculas e devolve o resultado ao cliente
- 4) O Cliente lê da `socket` (`stream inFromServer`) a linha modificada e imprime-a

NOTA: Neste caso, uma mensagem é uma linha de texto, e o terminador de mensagem é o carácter de mudança de linha

Exemplo: cliente TCP em Java

```
import java.io.*;
import java.net.*;
class TCPClient {

    public static void main(String argv[]) throws Exception {
        String sentence;
        String modifiedSentence;
```

Cria stream
de entrada

```
        BufferedReader inFromUser =
            new BufferedReader(new InputStreamReader(System.in));
```

Cria socket do cli-
ente e estabelece,
conexão c/ servidor;
DNS lookup implícito

```
        Socket clientSocket = new Socket("hostname", 6789);
```

Cria stream de
saída associada
à socket

```
        DataOutputStream outToServer =
            new DataOutputStream(clientSocket.getOutputStream());
```

Exemplo: cliente TCP em Java (cont.)

Cria stream de
entrada asso-
ciada à socket

```
BufferedReader inFromServer =  
    new BufferedReader(new  
        InputStreamReader(clientSocket.getInputStream()));
```

Bloqueia até ler
uma linha

```
sentence = inFromUser.readLine();
```

Terminador
de mensagem

Envia linha (pedido)
ao servidor

```
outToServer.writeBytes(sentence + '\n');
```

Lê linha (resposta)
do servidor

```
modifiedSentence = inFromServer.readLine();
```

```
System.out.println("FROM SERVER: " + modifiedSentence);
```

```
clientSocket.close();
```

```
}  
}
```

Exemplo: servidor TCP em Java

```
import java.io.*;  
import java.net.*;
```

```
class TCPServer {
```

```
    public static void main(String argv[]) throws Exception {  
        String clientSentence;  
        String capitalizedSentence;
```

Cria socket de
recepção à escuta
na porta 6789

```
        ServerSocket welcomeSocket = new ServerSocket(6789);
```

```
        while(true) {
```

Ciclo infinito (esperar por um
cliente e atendê-lo, repetir...)

```
            Socket connectionSocket = welcomeSocket.accept();
```

```
            BufferedReader inFromClient =
```

```
                new BufferedReader(new  
                    InputStreamReader(connectionSocket.getInputStream()));
```

Espera que um cli-
ente se conecte à
socket de recepção;
cria uma nova socket
para a comunicação

Cria stream de entra-
da associada à socket
de conexão

Exemplo: servidor TCP em Java (cont.)

Cria stream de
saída associada à
socket de conexão

```
DataOutputStream outToClient =  
    new DataOutputStream(  
        connectionSocket.getOutputStream());
```

Lê linha (pedido) da
socket de conexão

```
clientSentence = inFromClient.readLine();
```

Escreve linha
(resposta) na
socket de conexão

```
capitalizedSentence = clientSentence.toUpperCase() + '\n';
```

```
outToClient.writeBytes(capitalizedSentence);
```

```
connectionSocket.close();
```

```
}  
}  
}
```

Fim do ciclo *while*:
regressa ao princípio do ciclo
para esperar por uma nova conexão

Tipos de servidores TCP

❑ Iterativo (exemplo anterior)

- ❖ Os pedidos são processados **sequencialmente**
 - Só depois de acabar de responder a um pedido é possível começar a responder a outro
- ❖ Só é utilizável em serviços que recebam pedidos **esporádicos** e **de resposta imediata** (e.g., daytime)

❑ Concorrente

- ❖ Processos
 - Sempre que o servidor recebe um novo pedido de conexão lança um **novo processo** (`fork()`) para o atender, enquanto o processo-pai continua à escuta
 - Funciona bem em serviços em que os **pedidos podem demorar muito a atender** mas não são demasiado frequentes (custo do `fork()`)

Tipos de servidores TCP

❑ Concorrente (cont.)

❖ Threads

- Semelhante ao concorrente com processos, mas em vez de um processo é lançada uma thread
- Mais leve em termos de utilização de CPU, mas menos robusto

❖ Multiplex

- Um único processo/thread usa `select()` para atender pedidos de **múltiplas sockets** em simultâneo
- Permite relacionar diferentes clientes (e.g., chat)
- Permite escutar em **múltiplas portas**
- Usa menos memória que outros tipos de servidor concorrente
- Implementação mais complexa
- Não tira partido de CPUs multi-core

Tipos de servidores TCP

❑ Pre-forking

- ❖ Logo **no arranque**, o servidor lança um certo número de **processos** para atender pedidos em paralelo
- ❖ Pode haver ou não um processo coordenador que distribui a carga pelos restantes
- ❖ Funciona bem com **cargas elevadas**, nomeadamente com um número muito elevado de pedidos curtos (e.g., servidor HTTP)

❑ Pre-threaded

- ❖ Semelhante ao pre-forking, mas com threads em vez de processos
- ❖ Ligeiramente mais eficiente*, mas menos robusto

Capítulo 2: Camada de aplicação

- ❑ 2.1 Princípios das aplicações em rede
- ❑ 2.2 Web e HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Correio electrónico
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS
- ❑ 2.6 Aplicações P2P
- ❑ 2.7 Programação com *sockets* — TCP
- ❑ 2.8 Programação com *sockets* — UDP

Programação com sockets: UDP

UDP: não há estabelecimento de conexão entre cliente e servidor

- ❑ não há *handshaking*
- ❑ o emissor especifica o endereço IP e a porta de destino de cada pacote enviado
- ❑ o servidor extrai o endereço IP e a porta de origem do pacote com o pedido para saber para onde enviar a resposta

UDP: a informação transmitida pode perder-se, ser recebida fora de ordem ou mesmo ser duplicada

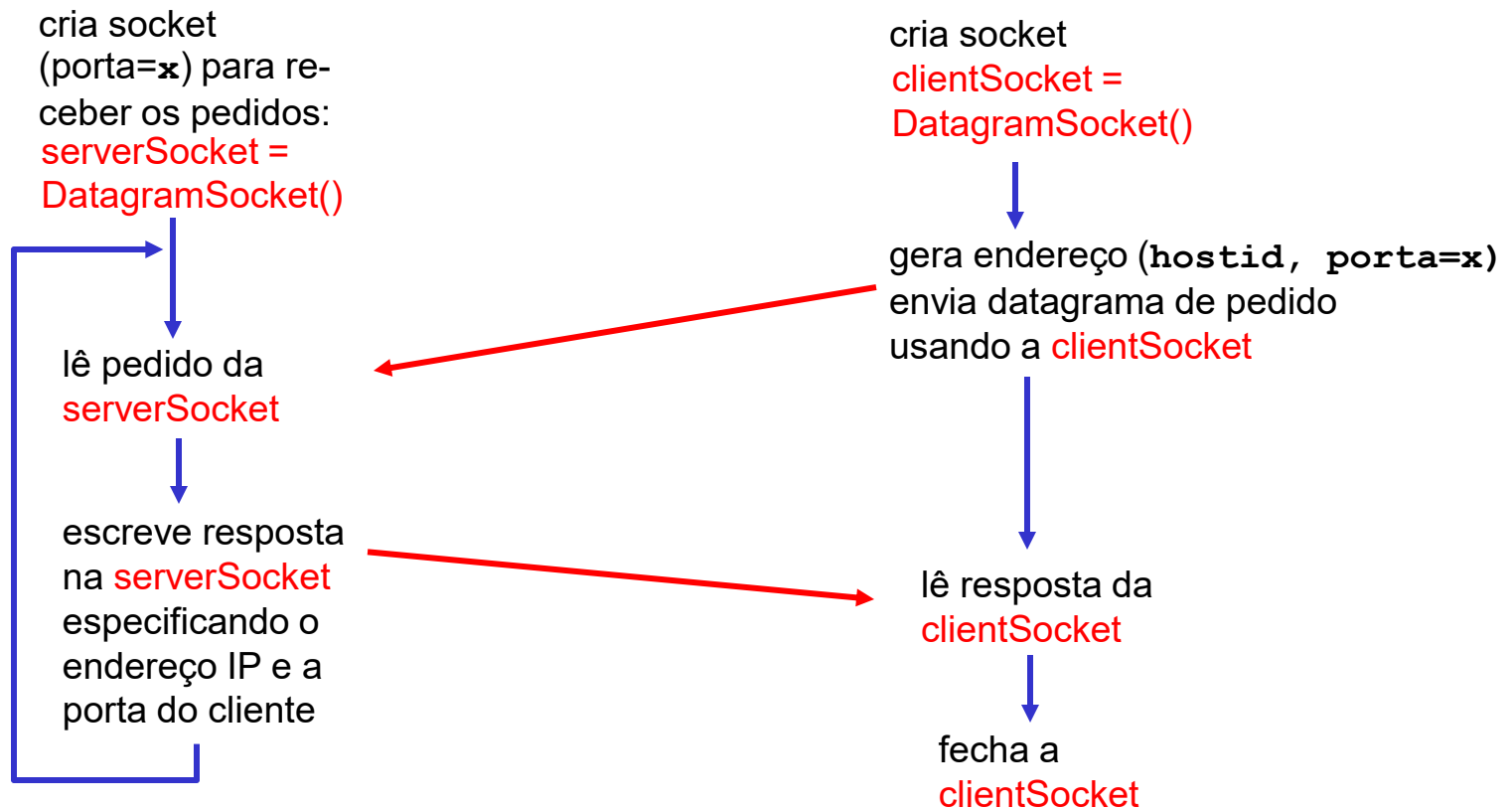
para a aplicação

o UDP fornece um serviço de transferência não-fiável de blocos de informação ("datagramas") entre o cliente e o servidor

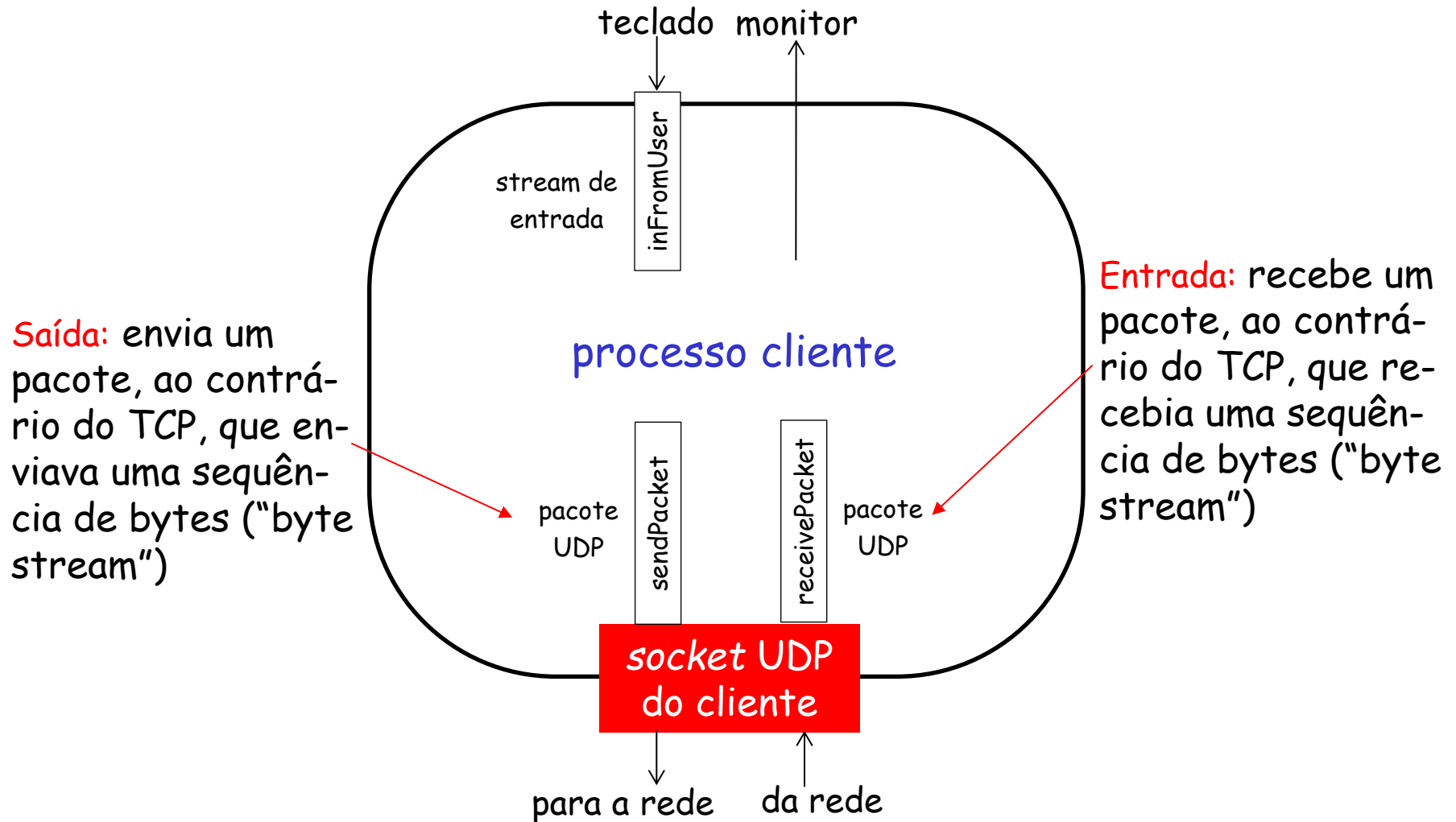
Interação cliente/servidor: UDP

Servidor (a correr em `hostid`)

Cliente



Exemplo: cliente UDP em Java



Exemplo: cliente UDP em Java

```
import java.io.*;
import java.net.*;
```

```
class UDPClient {
    public static void main(String args[]) throws Exception {
```

Cria stream de entrada → `BufferedReader inFromUser = new BufferedReader(new InputStreamReader(System.in));`

Cria socket do cliente → `DatagramSocket clientSocket = new DatagramSocket();`

Traduz nome para endereço IP usando o DNS → `InetAddress IPAddress = InetAddress.getByName("hostname");`

```
String sentence = inFromUser.readLine();
byte[] sendData = sentence.getBytes();
```

Exemplo: cliente UDP em Java (cont.)

Cria datagrama com dados a enviar, comprimento, endereço IP e porta

```
DatagramPacket sendPacket =  
    new DatagramPacket(sendData, sendData.length,  
                        IPAddress, 9876);
```

Envia datagrama (pedido) ao servidor

```
clientSocket.send(sendPacket);
```

```
byte[] receiveData = new byte[1024];  
DatagramPacket receivePacket =  
    new DatagramPacket(receiveData, receiveData.length);
```

Lê datagrama (resposta) do servidor, bloqueando até que chegue

```
clientSocket.receive(receivePacket);
```

```
String modifiedSentence =  
    new String(receivePacket.getData());
```

```
System.out.println("FROM SERVER:" + modifiedSentence);  
clientSocket.close();
```

```
}  
}
```

Exemplo: servidor UDP em Java

```
import java.io.*;
import java.net.*;
```

```
class UDPServer {
    public static void main(String args[]) throws Exception {
```

Cria socket de datagramas na porta 9876

```
        DatagramSocket serverSocket = new DatagramSocket(9876);
```

```
        byte[] receiveData = new byte[1024];
        byte[] sendData;
```

```
        while(true) {
```

Cria estrutura de dados para receber o datagrama

```
            DatagramPacket receivePacket =
                new DatagramPacket(receiveData, receiveData.length);
```

Recebe datagrama vindo do cliente

```
            serverSocket.receive(receivePacket);
```

Exemplo: servidor UDP em Java (cont.)

```
String sentence = new String(receivePacket.getData());
```

Obtém endereço
IP e porta do
emissor (cliente)

```
InetAddress IPAddress = receivePacket.getAddress();
```

```
int port = receivePacket.getPort();
```

```
String capitalizedSentence = sentence.toUpperCase();
```

Cria datagrama a
enviar p/ o cliente

```
sendData = capitalizedSentence.getBytes();
```

Escreve data-
grama a enviar
pela socket

```
DatagramPacket sendPacket =  
    new DatagramPacket(sendData, sendData.length, IPAddress,  
                        port);
```

```
serverSocket.send(sendPacket);
```

```
}  
}  
}
```

Fim do ciclo *while*: regressa ao princípio do ciclo
para esperar por um novo datagrama

Capítulo 2: Sumário

e assim se completa o nosso estudo da camada de aplicação

- arquiteturas de aplicação
 - ❖ cliente-servidor
 - ❖ P2P
 - ❖ híbridas
- requisitos de serviço por parte das aplicações:
 - ❖ fiabilidade, débito, atraso
- Modelos do serviço de transporte na Internet
 - ❖ fiável, orientado a conexões: TCP
 - ❖ não fiável, transporte de datagramas: UDP
- protocolos específicos:
 - ❖ HTTP
 - ❖ FTP
 - ❖ SMTP, POP, IMAP
 - ❖ DNS
 - ❖ P2P: BitTorrent
- programação com sockets

Capítulo 2: Sumário

O mais importante: aprendemos sobre os protocolos

□ troca de mensagens típica pedido/resposta:

- ❖ cliente pede informação ou serviço
- ❖ servidor responde com dados e/ou código de estado

□ formato das mensagens:

- ❖ cabeçalhos: campos que fornecem informação sobre os dados
- ❖ dados: informação comunicada

Aspectos importantes:

- mensagens de controlo e de dados
 - ❖ in-band, out-of-band
- centralizado ou distribuído
- com ou sem manutenção de estado
- transferência de mensagens fiável ou não-fiável
- complexidade nas extremidades da rede